



UNIVERSIDAD
DE MURCIA

Departamento
de Informática
y Sistemas

TRABAJO FIN DE GRADO

MovieMate: app de gestión y valoración de películas y series

Murcia, a XX de XXXXXXX de 20XX

Christian Matas Conesa
24427423C
christian.m.c@um.es

Francisco García Sánchez

ÍNDICE DE CONTENIDOS

I.	Resumen.....	5
II.	Extended abstract	6
1	Introducción	11
2	Estado del arte	12
2.1	Análisis de la situación de partida	12
2.2	Tecnologías empleadas.....	15
2.2.1	Back-end	15
2.2.2	Front-end.....	16
2.2.3	Integración de datos.....	17
2.2.4	Miscelánea.....	17
3	Análisis de objetivos y metodología.....	19
3.1	Objetivos.....	19
3.2	Metodología.....	19
4	Diseño y resolución del trabajo realizado	22
4.1	Análisis	22
4.1.1	Usuarios no registrados.....	22
4.1.2	Usuarios registrados.....	23
4.2	Diseño	26
4.2.1	Modelo de datos	26
4.2.2	Diseño del back-end	30
4.2.3	Diseño del front-end.....	34
4.3	Implementación.....	35
4.3.1	Arquitectura del back-end.....	37
4.3.2	Capa de presentación (Controladores)	39
4.3.3	Arquitectura del front-end	45
4.4	Pruebas	50
4.5	Despliegue	51
4.5.1	Contenerización con Docker.....	51
4.5.2	Orquestación con Docker Compose	52
4.5.3	Integración continua y automatización (CI/CD)	52
4.5.4	Despliegue en Kubernetes con Minikube.....	53
5	Conclusiones y vías futuras	56
5.1	Conclusiones	56

5.2	Trabajo futuro.....	56
6	Bibliografía	58
Anexo A – Especificación de casos de uso		60
A.1	Usuarios no registrados	60
A.2	Interacción social	61
A.3	Listas personalizadas.....	63
A.4	Valoraciones y reseñas.....	65

ÍNDICE DE FIGURAS

Figura 1. Aplicación web Letterboxd.....	13
Figura 2. Aplicación web IMDb.....	13
Figura 3. Aplicación web Serializd	14
Figura 4. Diagrama de casos de uso de un usuario no registrado	22
Figura 5. Diagrama de casos de uso de la interacción social de un usuario registrado.....	23
Figura 6. Diagrama de casos de uso sobre listas personalizadas de un usuario registrado	24
Figura 7. Diagrama de casos de uso de valoraciones y reseñas de un usuario registrado	25
Figura 8. Diagrama entidad-relación de MovieMate	27
Figura 9. Arquitectura de MovieMate.....	35
Figura 11. Estructura de carpetas del back-end de MovieMate.....	38
Figura 12. Ejemplo de controlador REST (FollowerController) en MovieMate	40
Figura 13. Ejemplo de servicio (FollowerService) en MovieMate.....	41
Figura 14. Ejemplo de repositorio JPA (FollowerRepository) en MovieMate.....	42
Figura 15. Configuración de seguridad.....	43
Figura 17. Endpoints para gestión de seguidores en MovieMate	44
Figura 16. Ejemplo DTO.....	45

ÍNDICE DE TABLAS

Tabla 1. Resumen comparativo de las soluciones analizadas.....	15
Tabla 2. Plan de trabajo: planificación mensual de las tareas	20
Tabla 3. Plan de trabajo: estimación de horas por tarea.....	21
Tabla 4. <i>Endpoints</i> de la aplicación	30

I. Resumen

Este es el resumen...

ESTILO DE REDACCIÓN APLICABLE A TODO EL DOCUMENTO: evitaremos el uso de la primera persona; en su lugar se suele emplear el reflexivo (“hemos desarrollado” → “se ha desarrollado”).

MovieMate es una plataforma web social para la gestión, valoración y descubrimiento de contenido audiovisual, diseñada para cubrir las carencias detectadas en las herramientas existentes de referencia. El proyecto surge de la ausencia de una solución unificada que permita a los aficionados al cine y las series gestionar su historial de visionado, emitir valoraciones, organizar listas personalizadas e interactuar socialmente con otros usuarios en un único espacio.

Se ha desarrollado una aplicación web full-stack que integra un back-end implementado con Spring Boot y Java, siguiendo una arquitectura en capas basada en el patrón MVC, con persistencia relacional en PostgreSQL y autenticación sin estado mediante JSON Web Tokens. El front-end se ha desarrollado con React y TypeScript, adoptando una arquitectura basada en componentes con gestión del estado asíncrono mediante TanStack Query y estado global mediante Zustand.

La plataforma se integra con la API de The Movie Database (TMDb) para obtener información actualizada sobre películas y series, implementando una estrategia de caché bajo demanda que minimiza las llamadas a servicios externos y optimiza el rendimiento del sistema. El despliegue se realiza de forma contenerizada mediante Docker y Docker Compose, y el proyecto dispone de un pipeline de integración continua implementado con GitHub Actions.

El resultado es un prototipo funcional de una red social cinematográfica que demuestra la viabilidad técnica de integrar funcionalidades sociales, valoraciones, sincronización con fuentes externas de datos y comunicación en tiempo real en un stack tecnológico moderno y bien estructurado.

II. Extended abstract

Resumen extendido en inglés (2000 palabras). En el abstract conviene replicar de algún modo la estructura de la memoria en sí: (i) introducción, (ii) estado del arte, (iii) objetivos y metodología, (iv) diseño y resolución del trabajo, (v) conclusiones y trabajo futuro. El contenido del abstract tiene que abarcar todos esos conceptos en ese orden.

Context and Motivation

The consumption of audiovisual content has grown exponentially in recent years, driven by the proliferation of streaming platforms and the widespread availability of high-speed internet access. In this context, tools dedicated to cataloguing and rating films and television series — such as Letterboxd, IMDb, or Serializd — have become indispensable companions for enthusiasts who wish to keep track of their viewing history, share opinions with other users, and discover new content within a community. However, these platforms present significant limitations that fragment the user experience: most specialise in a single content type (either films or series, but not both simultaneously), lack advanced social features, or do not provide a unified space in which a user can manage ratings, reviews, and personal lists together. This situation creates a suboptimal experience for those who consume both cinema and television and desire a single, integral platform to centralise their entire audiovisual activity.

MovieMate was conceived to address these shortcomings. It is a full-stack social web application that combines the content cataloguing capabilities of IMDb, the personal diary and community features of Letterboxd, and the series tracking functionality of Serializd into a single, coherent system. The platform allows users to rate and review both films and television series, organise content into custom lists, follow other users and track their activity, receive real-time notifications, and obtain personalised content recommendations based on their viewing history.

Objectives

The primary objective of this Bachelor's thesis is the design and development of a responsive, full-stack web application for the consultation, management, and rating of films and television series. This overarching goal is decomposed into the following specific sub-objectives: to identify the most appropriate technologies and tools for the undertaking; to determine the main features and shortcomings of existing solutions in the domain; to develop an application that addresses those shortcomings while satisfying the principal quality standards of modern software engineering; to deploy the application in a production-like environment; and to document the work carried out thoroughly. It is important to note that the project aims to deliver a functional first prototype and does not intend to address content distribution or streaming. External data are sourced from The Movie Database (TMDB) API to populate the system with real film and series metadata.

Methodology

The project followed an iterative and incremental development methodology organised into five sequential phases. The first phase consisted of a preliminary analysis of existing platforms and an evaluation of candidate technologies for both the server-side and client-side layers. The

second phase covered system analysis and design: establishing functional and non-functional requirements, designing the relational data model, specifying the REST API contracts, and producing interface mockups. The third phase comprised the implementation of both the back-end and the front-end components. The fourth phase was devoted to verification and validation through automated unit testing and a comprehensive smoke-test script covering all API endpoints. The fifth and final phase addressed the documentation of the system and the preparation of this report.

Throughout development, Git was used for version control following a two-branch strategy: all active development took place on a dedicated develop branch, and merges into main were only performed once the automated CI/CD pipeline had completed all verification steps successfully. This approach ensured that the main branch always represented a stable, deployable version of the application.

Back-End Architecture and Implementation

The server-side component of MovieMate was implemented using Spring Boot 3.3.4 on Java 21, with PostgreSQL 16 as the relational database management system. The architecture follows a classic layered pattern divided into four tiers: the presentation layer (REST controllers), the business logic layer (services), the persistence layer (Spring Data JPA repositories), and the security layer.

Security is handled by Spring Security in a stateless configuration. Authentication is based on JSON Web Tokens (JWT): upon successful login, the server issues a signed token with a 24-hour expiry that the client includes in the Authorization header of every subsequent request. Passwords are stored using BCrypt hashing. Access control distinguishes between unauthenticated users, regular authenticated users, and administrators, with each endpoint protected accordingly.

Content metadata — film and series information, cast, genres, and ratings — is sourced from The Movie Database API. Rather than pre-loading the entire TMDB catalogue, MovieMate adopts a lazy synchronisation strategy: content is fetched from TMDB on demand the first time a user interacts with a title and cached in the local database. A TTL-based refresh mechanism ensures that frequently accessed or recently added titles remain up to date, while less popular content is refreshed on a longer schedule. This approach significantly reduces dependency on the external API and improves response times for repeated requests.

The data model encompasses fifteen entities: User, Content, Rating, List, ListContent, ListComment, Follower, FollowRequest, Notification, Activity, Comment, ReviewLike, UserStats, UserBadge, and EpisodeWatch. The REST API exposes over seventy endpoints grouped into twelve controllers — authentication, users, content, ratings, lists, comments, followers, notifications, activity, TMDB, admin, and episode tracking — all documented via Swagger/OpenAPI.

Real-time notifications are delivered over WebSocket using the STOMP messaging protocol over SockJS. The server authenticates WebSocket connections by inspecting the JWT token included in the STOMP CONNECT frame through a custom WebSocketAuthInterceptor. When

an event occurs — a new follower, a like on a review, a comment on a rating, or a comment on a list — the server pushes a notification to the corresponding user's private channel, which the client receives and renders without polling.

Additional back-end features include a gamification subsystem with ten badge types awarded automatically based on user activity milestones, a personalised recommendation engine that derives the user's top genre from their rating history and queries TMDB's discovery endpoint for highly rated titles in that genre, an episode-level tracking system for television series, a content reporting and moderation system managed through the admin panel, and a DataSeeder component that populates a fresh deployment with a realistic set of test users, content, ratings, and social connections.

Front-End Architecture and Implementation

The client-side application was developed as a Single Page Application (SPA) using React 19 and TypeScript, bundled with Vite 7. The code is organised following a feature-based directory structure in which each functional module — lists, profile, discover, notifications, and others — groups its own pages, components, hooks, and API access logic, minimising coupling between modules.

Global synchronous state — primarily the authenticated user's session data and JWT token — is managed with Zustand. Server-derived asynchronous state is managed with TanStack Query v5, which handles the full lifecycle of HTTP requests: caching results under a structured key hierarchy, automatically revalidating stale data, and performing selective cache invalidation after mutations. All HTTP communication is channelled through a global Axios instance configured with a request interceptor that attaches the JWT Authorization header to every authenticated request.

Navigation is handled by React Router v6 with a centralised route declaration in App.tsx. Routes are divided into public routes accessible without authentication and protected routes guarded by a PrivateRoute component that redirects unauthenticated visitors to the login screen.

The application comprises the following main pages. The home page presents trending content sourced from TMDB, a personalised recommendations carousel labelled "For You", and a section of suggested users to follow. The discovery page provides a debounced search bar that queries both films and series simultaneously, interleaving results by relevance. The content detail page — the most complex view — displays the backdrop image, poster, metadata, a five-star rating widget that supports both creation and in-place editing of existing ratings, a dialogue for adding the title to any personal list, the cast gallery, and for television series an episode tracking accordion. The profile page is organised into four tabs: activity timeline, lists, advanced statistics including rating distribution by score value and top genres, and earned badges. The lists pages allow users to create, edit, and delete custom lists, browse their contents, and leave public comments. The activity page displays a chronological feed of actions performed by followed users. Real-time notifications are reflected immediately in the navigation badge through the WebSocket connection established by the root layout component.

The visual identity follows a dark theme defined through Tailwind CSS 4 custom tokens: a deep navy-to-charcoal background scale, a golden accent colour (#e8c97a) used for all interactive elements, and a dual typography system pairing Playfair Display for titles with DM Sans for body text. Accessible components from Radix UI and Shadcn/ui ensure compatibility with assistive technologies. The interface is fully responsive: a fixed bottom navigation bar replaces the sidebar on mobile viewports, and all content grids adapt fluidly to any screen size.

Testing and Deployment

The quality of the server-side code is validated by a suite of 205 automated unit tests written with JUnit 5 and Mockito, organised into seventeen test classes that collectively cover all service layer components. Tests follow the Arrange-Act-Assert pattern and make extensive use of mock objects to isolate the unit under test from its dependencies. A complementary shell-based smoke-test script exercises all API endpoints in an integrated fashion against a running instance of the application.

The back-end is packaged using a multi-stage Docker build: the first stage compiles the project with Maven on a Java 21 image and produces the executable JAR; the second stage copies the JAR into a minimal Eclipse Temurin JRE 21 runtime image, reducing the final image size and eliminating build-time dependencies. The full system is orchestrated with Docker Compose, which brings up the back-end and the PostgreSQL 16 database with a single command. Continuous integration is implemented via GitHub Actions, triggering on every push to the develop branch and on pull requests targeting main; it executes the full test suite, builds the Maven artefact, and verifies code style with Checkstyle. A successful merge to main additionally triggers the Docker image build job. Kubernetes manifests have been defined for a production-grade deployment, encompassing a dedicated namespace, ConfigMaps and Secrets for configuration and credentials, PersistentVolumeClaims for database storage, Deployments with liveness and readiness probes, and ClusterIP/NodePort services. Local validation of the Kubernetes configuration is supported through Minikube, enabling deployment to a single-node cluster on the developer's machine without cloud infrastructure.

Results and Conclusions

MovieMate delivers a fully functional social platform for audiovisual content management that meets all the objectives established at the outset of the project. The application successfully integrates content discovery, personal cataloguing, social interaction, real-time communication, and personalised recommendations in a single, coherent system accessible from any device. From a technical standpoint, the project demonstrates the practical application of a broad set of modern web development technologies and patterns: RESTful API design, stateless JWT-based security, on-demand external API synchronisation, reactive client-side state management, WebSocket-based real-time communication, containerised deployment, and automated CI/CD pipelines. The layered back-end architecture and the feature-based front-end organisation proved effective choices at this scale, promoting maintainability and separation of concerns throughout the development lifecycle.

Several avenues for future work have been identified. The recommendation engine could be enhanced with collaborative filtering algorithms or machine learning techniques to provide

more accurate and personalised suggestions. A native mobile application for iOS and Android — leveraging React Native to reuse existing client logic — would significantly broaden the platform's reach. Integration with streaming service availability APIs would allow users to see, in real time, on which subscribed platforms a given title can be watched. Finally, at scale, a migration towards a microservices architecture would enable independent horizontal scaling of high-demand components such as the notification service and the TMDB synchronisation pipeline.

1 Introducción

El consumo de contenido audiovisual ha experimentado un crecimiento exponencial en los últimos años, impulsado por la proliferación de plataformas de streaming y por la democratización del acceso a internet de alta velocidad. En este contexto, herramientas de gestión y valoración de contenido como Letterboxd [1], IMDb [2] o Serializd [3] han ganado popularidad como complementos indispensables para los aficionados al cine y las series, permitiéndoles llevar un registro de sus visionados, compartir opiniones y descubrir nuevo contenido en comunidad.

Sin embargo, estas plataformas presentan limitaciones que fragmentan la experiencia del usuario. La mayoría se especializan en un único tipo de contenido (películas o series, pero no ambos), carecen de funcionalidades sociales avanzadas o no permiten la gestión unificada de listas y valoraciones en un único espacio. Esta situación genera una experiencia subóptima para quienes consumen tanto cine como televisión y desean una plataforma integral que centralice su actividad audiovisual.

Ante esta situación, el presente Trabajo de Fin de Grado propone el diseño y desarrollo de MovieMate, una plataforma social para la gestión y valoración de películas y series que integra, en un único sistema, las funcionalidades más demandadas de las herramientas existentes. MovieMate permite a sus usuarios llevar un registro completo de su historial de visionado, emitir valoraciones y reseñas, organizar su contenido en listas personalizadas y seguir la actividad de otros usuarios, conformando una red social temática centrada en el contenido audiovisual.

Desde el punto de vista técnico, el proyecto constituye una oportunidad para aplicar de forma integrada conocimientos adquiridos a lo largo del grado en áreas como el desarrollo de aplicaciones web full-stack, el diseño de bases de datos relacionales, la seguridad en aplicaciones mediante autenticación basada en tokens, la integración de servicios externos mediante APIs REST y el despliegue de sistemas contenerizados en entornos de producción.

El resto del documento se organiza de la siguiente manera. El Capítulo 2 analiza el estado del arte, estudiando las plataformas existentes de referencia y las tecnologías empleadas en el desarrollo. El Capítulo 3 describe los objetivos del proyecto y la metodología de trabajo seguida. El Capítulo 4 aborda el diseño y la implementación del sistema, incluyendo el modelo de datos, la arquitectura del back-end y del front-end, las pruebas realizadas y el proceso de despliegue. Finalmente, el Capítulo 5 recoge las conclusiones del trabajo y las líneas de trabajo futuro identificadas.

Esto se escribe lo último, junto con el resumen y el abstract. Se establece el contexto en el que se sitúa el proyecto introduciendo claramente la problemática e indicando el objetivo general del proyecto (como consecuencia de esos problemas que se quieren resolver). Suele venir acompañado de numerosas referencias bibliográficas relevantes sobre los distintos conceptos tratados. En el último párrafo de la introducción hay que indicar la estructura/organización del resto del documento (un párrafo indicando brevemente en qué secciones se ha dividido el documento y el contenido de cada sección).

2 Estado del arte

En este capítulo se analizará el contexto previo al desarrollo de **MovieMate**, detallando las aplicaciones y soluciones existentes para la organización, consulta y valoración de películas y series. Además, se describirán las tecnologías que se ha decidido emplear para la realización del proyecto.

Formatted: Font: Bold

2.1 Análisis de la situación de partida

Actualmente, existen numerosas aplicaciones y plataformas web destinadas a la consulta, organización y valoración de películas y series. La digitalización del consumo audiovisual y el auge de los servicios de *streaming* han impulsado la demanda de herramientas que permitan a los usuarios gestionar sus listas personales, descubrir nuevo contenido y compartir sus opiniones [1].

Para este análisis, se han seleccionado tres aplicaciones representativas: **Letterboxd**, **IMDb** y **Serializd**. La elección de estas plataformas se ha basado en su relevancia y popularidad en el sector, confirmada por su presencia destacada en rankings de aplicaciones y artículos especializados **Error! Reference source not found.**, [2]. Además, se ha buscado una diversidad de enfoques: Letterboxd se centra en el aspecto social y la crítica comunitaria, IMDb actúa como una base de datos global con un componente social secundario, y Serializd se especializa específicamente en el seguimiento de series de televisión, lo que permite obtener una visión amplia de las opciones disponibles para los usuarios.

En la evaluación, se han considerado las siguientes características clave, justificadas por ser los pilares fundamentales del objetivo de MovieMate:

- **Gestión de listas personales:** Capacidad para crear, editar y organizar listas personalizadas de películas y series (p. ej., "Favoritas", "Por ver").
- **Sistema de valoración y diario:** Funcionalidades para puntuar contenido y registrar la fecha de visualización (diario personal).
- **Enfoque de red social:** Capacidad para seguir a otros usuarios, comentar sus actividades y descubrir contenido a través de su actividad.
- **Experiencia de usuario y diseño:** Facilidad de uso, diseño visual, claridad en la presentación de la información y adaptabilidad a diferentes dispositivos (diseño *responsive*).

Letterboxd [3] (ver Figura 1) es una red social centrada en el cine que permite a los usuarios registrar las películas que ven, crear listas personalizadas, escribir críticas y seguir la actividad de otros cinéfilos. Su interfaz es limpia y minimalista, priorizando el contenido generado por la comunidad. Su sistema de "diario" es una de sus características más valoradas, permitiendo un registro cronológico personal de las visualizaciones. Sin embargo, su punto débil más notable es la falta de un enfoque equivalente para las series de televisión, un vacío que otras aplicaciones intentan llenar.

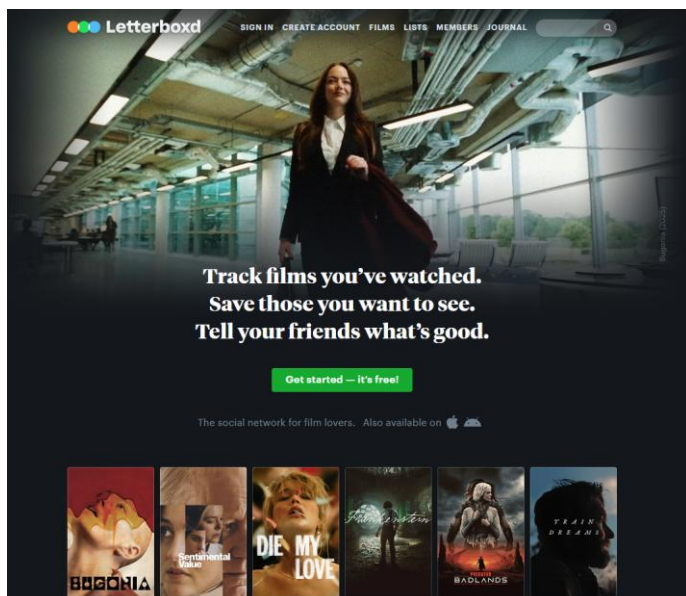


Figura 1. Aplicación web Letterboxd

IMDb (Internet Movie Database) [5] es una de las bases de datos en línea más grandes y populares sobre cine y televisión. Además de proporcionar información técnica y artística exhaustiva, permite a los usuarios puntuar títulos, escribir críticas y mantener una lista personal de "Watchlist". Si bien su componente social existe, está menos desarrollado que en Letterboxd, y su interfaz puede resultar sobrecargada de información (ver Figura 2) para el usuario que busca una experiencia simple de gestión y seguimiento.

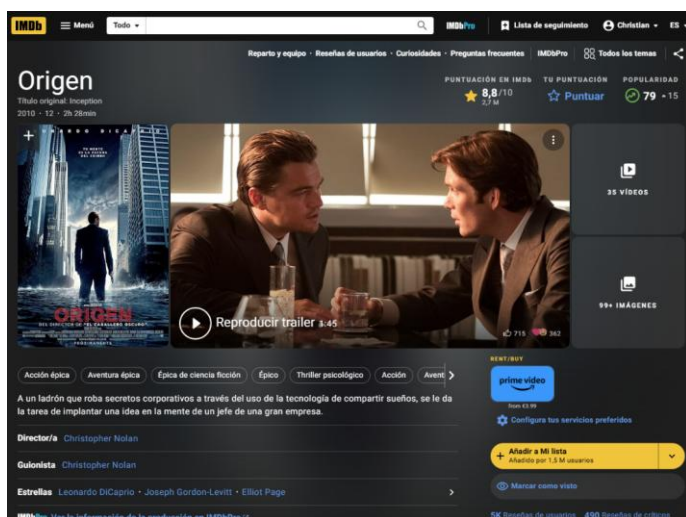


Figura 2. Aplicación web IMDb

Serialized [6] es una aplicación relativamente nueva (2023) que adopta el modelo y la filosofía de Letterboxd pero aplicada exclusivamente al mundo de las series de televisión. Los usuarios pueden marcar episodios vistos, puntuar temporadas completas o episodios individuales, y seguir a otros usuarios para descubrir nuevas series (ver Figura 3). Su principal limitación es, precisamente, su especialización, lo que obliga a los usuarios interesados tanto en cine como en televisión a utilizar dos aplicaciones diferentes.

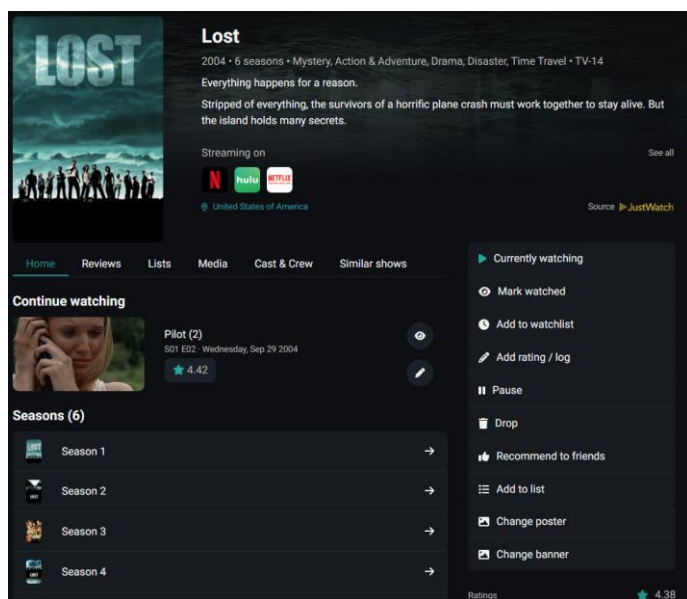


Figura 3. Aplicación web Serialized

La aplicación desarrollada en este TFG, **MovieMate**, pretende integrar las mejores características de las soluciones analizadas, ofreciendo una experiencia unificada para la gestión tanto de películas como de series. Propone una visión centrada en la usabilidad y la interacción social, con un diseño intuitivo y adaptable. En la Tabla 1 se muestra un resumen comparativo de las soluciones analizadas.

Tabla 1. Resumen comparativo de las soluciones analizadas

Aplicación	Gestión de Listas	Valoración y Diario	Red Social	Experiencia de Usuarios	Cine/ Series
Letterboxd	Listas personalizadas, watchlist, vistas	Diario personal, puntuación por película, reseñas	Seguir usuarios, comentarios, likes	Interfaz minimalista	Solo Cine
IMDb	Watchlist básica, listas personalizadas limitadas	Puntuación global, reseñas de usuarios	Sistema de reseñas, perfil básico	Interfaz funcional pero sobrecargada de información	Cine y series
Serializd	Listas personalizadas, seguimiento por episodios	Puntuación por episodio/temporada, diario de visionado	Seguir usuarios, comentarios en episodios	Interfaz inspirada en Letterboxd	Solo Series
MovieMate	Listas personalizadas completas, watchlist, categorías múltiples	Diario personalizado, puntuación flexible, reseñas	Seguir usuarios, comentarios, reseñas	Interfaz unificada e intuitiva, diseño adaptable	Cine y series

2.2 Tecnologías empleadas

En este apartado se detallan los lenguajes de programación, *frameworks*, librerías y otras herramientas que se planea utilizar en el desarrollo de MovieMate, abarcando tanto el lado del servidor (*back-end*) como el del cliente (*front-end*).

2.2.1 Back-end

Para el desarrollo del servidor, se ha elegido el **framework Spring Boot** [7] junto con el lenguaje de programación **Java** [8]. Spring Boot es un *framework* que simplifica enormemente el desarrollo de aplicaciones basadas en Spring, proporcionando configuración automática y permitiendo crear aplicaciones listas para producción con mínimo esfuerzo. Java, como lenguaje de programación orientado a objetos, robusto y con un amplio ecosistema, garantiza escalabilidad, mantenibilidad y alto rendimiento, siendo una elección muy extendida en el desarrollo de *enterprise applications*.

Como sistema de gestión de bases de datos se utilizará **PostgreSQL** [9]. Se ha optado por esta base de datos relacional de código abierto debido a su robustez, cumplimiento de estándares SQL y su excelente rendimiento para cargas de trabajo complejas. PostgreSQL ofrece características avanzadas como tipos de datos complejos, consultas JSON nativas y una fuerte consistencia transaccional, lo que la hace ideal para gestionar las relaciones estructuradas entre usuarios, películas, series, listas y valoraciones en MovieMate.

Commented [FG1]: Quizás merezca la pena extender esto un poco relatando, por ejemplo, las librerías concretas de Spring Boot utilizadas en el ámbito del desarrollo (esto lo puedes ir introduciendo conforme vayas avanzando con el desarrollo); por ejemplo, para interaccionar con PostgreSQL o para la definición de la API REST.

Commented [FG2R1]: NO HECHO

La comunicación entre el *front-end* y el *back-end* se realizará a través de una **API RESTful** [10], siguiendo los principios arquitectónicos REST para garantizar una interfaz coherente y sin estado.

En el desarrollo concreto del *back-end* se han utilizado distintos módulos del ecosistema de Spring Boot que facilitan la implementación de las distintas capas de la aplicación. Para la construcción de la API REST se ha empleado el módulo `spring-boot-starter-web`, que proporciona las funcionalidades necesarias para definir controladores mediante anotaciones y gestionar peticiones HTTP siguiendo el modelo MVC (Modelo-Vista-Controlador). De este modo, los distintos endpoints del sistema pueden exponerse de forma estructurada y coherente.

Para la persistencia de datos se ha utilizado `spring-boot-starter-data-jpa`, que implementa la especificación JPA (Java Persistence API) y permite realizar el mapeo objeto-relacional entre las entidades del dominio y las tablas de la base de datos. Esto facilita la abstracción del acceso a datos mediante repositorios, reduciendo la complejidad del código y mejorando su mantenibilidad. La conexión con PostgreSQL se realiza a través del *driver* oficial incluido como dependencia del proyecto.

En el ámbito de la seguridad se ha integrado `spring-boot-starter-security`, que proporciona los mecanismos de autenticación y autorización de la aplicación. Sobre esta base se ha implementado un sistema de autenticación basado en JSON Web Tokens (JWT), utilizando la librería JWT para la generación y validación de los tokens. Este enfoque permite mantener una arquitectura sin estado (*stateless*), adecuada para una API REST moderna.

Asimismo, se ha incorporado `spring-boot-starter-validation` para la validación de datos de entrada, garantizando la integridad de la información recibida por los *endpoints*. Para la documentación automática de la API se ha empleado SpringDoc OpenAPI, que genera una interfaz interactiva compatible con el estándar **OpenAPI** [11]/**Swagger** [12], facilitando la exploración y prueba de los servicios desarrollados.

Finalmente, se ha incluido el módulo `Actuator` para exponer información de monitorización y estado de la aplicación, así como dependencias específicas para pruebas automatizadas, incluyendo una base de datos en memoria **H2** [13] para el entorno de *testing* y herramientas de análisis de cobertura como **JaCoCo** [14].

2.2.2 **Front-end**

El *front-end* de la aplicación se implementará utilizando **React** [15] junto con **TypeScript** [16]. React es una biblioteca de JavaScript ampliamente adoptada que permite crear interfaces de usuario interactivas y basadas en componentes reutilizables. TypeScript, al añadir tipado estático, contribuye a la detección temprana de errores, mejora la mantenibilidad del código y proporciona una mejor experiencia de desarrollo, lo que es especialmente valioso en proyectos de mediana y gran escala.

Para el diseño de la interfaz de usuario y garantizar que sea completamente *responsive*, se empleará **Tailwind CSS** [17]. Tailwind es un *framework* de CSS de utilidad que permite construir diseños personalizados directamente en el marcado, agilizando el desarrollo de interfaces modernas y adaptables a cualquier tamaño de pantalla. Como complemento, se valorará el uso de una librería de componentes como **Mantine** [18] o **Shadcn/ui** para acelerar el desarrollo de elementos UI complejos y consistentes.

Para la gestión del estado de la aplicación se han combinado dos herramientas complementarias. Zustand [REF] se emplea como store global ligero para el estado sincrónico,

Commented [FG3]: Aquí algo parecido con librerías de React relevantes que vayas a emplear en el desarrollo.

Commented [FG4R3]: NO HECHO

Commented [CMCSR3]: Aún estoy implementando el frontend

como los datos del usuario autenticado y el token de sesión. TanStack Query [REF] gestiona el estado asíncrono derivado de peticiones al servidor, automatizando el ciclo de vida de las solicitudes, el almacenamiento en caché de resultados y la revalidación de datos cuando estos quedan desactualizados, lo que reduce considerablemente la complejidad del código de acceso a datos en los componentes.

La comunicación HTTP con el back-end se centraliza a través de Axios [REF], configurado con una instancia global que incorpora automáticamente las cabeceras de autorización en las peticiones autenticadas. La navegación entre páginas se gestiona mediante React Router v6 [REF], que soporta rutas anidadas y permite proteger las rutas que requieren sesión activa mediante componentes de guardia.

Los componentes de interfaz se construyen sobre Radix UI [REF] y su capa de estilos Shadcn/ui, que proporcionan elementos accesibles y personalizables compatibles con Tailwind CSS. Las notificaciones emergentes se implementan con la librería Sonner [REF]. Por último, para la comunicación en tiempo real se integra SockJS [REF] junto con el protocolo STOMP [REF], permitiendo al cliente recibir eventos del servidor de forma asíncrona a través de una conexión WebSocket persistente durante toda la sesión.

2.2.3 Integración de datos

Para poblar la base de datos de la aplicación con información actualizada de películas y series, se integrará **The Movie Database (TMDB)** [19]. TMDB es una API gratuita y muy completa que proporciona un extenso repositorio de metadatos, imágenes y puntuaciones sobre contenido audiovisual, siendo la opción estándar *de facto* para este tipo de proyectos.

2.2.4 Miscelánea

Además de las tecnologías mencionadas, se han empleado diversas herramientas que han facilitado el desarrollo y la gestión del proyecto:

- Visual Studio Code [20]: Editor de código fuente para el desarrollo.
- Git & GitHub [21], [22]: Para el control de versiones y alojamiento del repositorio.
- Postman [23]: Para la prueba y documentación de los *endpoints* de la API REST.
- pgAdmin 4 [24]: Para la gestión y visualización de las tablas de la base de datos.
- Dbeaver [25]: Para la elaboración del diagrama entidad-relación.
- ChatGPT [26]: Apoyo para la programación, elaboración de mockups, revisión de la arquitectura y documentación técnica.
- JUnit [27] & Mockito [28]: *Frameworks* utilizados para la implementación de pruebas unitarias del *back-end*, permitiendo validar la lógica de negocio de forma aislada mediante simulación de dependencias.
- Docker [29]: Plataforma de contenerización utilizada para empaquetar la aplicación *back-end* junto con sus dependencias en imágenes portables y reproducibles.
- Docker Compose [30]: Herramienta empleada para la orquestación de múltiples contenedores, permitiendo ejecutar conjuntamente el *back-end* y la base de datos PostgreSQL mediante una única configuración declarativa.

Commented [FG6]: Asegúrate de que todos los términos en inglés en el documento están en cursiva.

Formatted: Font: Italic

- GitHub Actions [31]: Servicio de integración continua utilizado para automatizar la ejecución de pruebas, construcción del proyecto y generación de la imagen Docker en cada actualización del repositorio.

3 Análisis de objetivos y metodología

En esta sección se describen los objetivos que se persiguen con el desarrollo de este proyecto, así como la metodología empleada (tareas y temporalización) para alcanzar estos objetivos.

3.1 Objetivos

El objetivo general que se plantea en este Trabajo Fin de Grado (TFG) es el desarrollo de una aplicación Web adaptable (responsiva) para la consulta, gestión y valoración de películas y series. Este objetivo se puede dividir en los siguientes subobjetivos específicos:

- (OB1) Identificar las tecnologías y herramientas más apropiadas para acometer el trabajo.
- (OB2) Determinar las principales propiedades y carencias de las soluciones actuales en el dominio de la aplicación.
- (OB3) Desarrollar una aplicación que subsane las carencias identificadas satisfaciendo los principales estándares de calidad en el desarrollo de software.
- (OB4) Desplegar la aplicación en un escenario de producción.
- (OB5) Documentar el trabajo realizado.

Es importante señalar que en este trabajo se persigue elaborar un primer prototipo de esta aplicación y no pretende abordar aspectos como distribución/streaming de contenido. Se integrarán APIs externas (TMDB), recopilando datos para poblar el prototipo. Por otro lado, también se establece como objetivo de este TFG la realización de una memoria en que se describan las principales características del trabajo realizado.

3.2 Metodología

A continuación, se enumeran las etapas en las que se ha dividido la realización del TFG. Para cada tarea se establecen las subtareas específicas y los objetivos abordados.

Tarea 1. Análisis preliminar

Objetivo(s): OB1, OB2

- 1.1. Consultar aplicaciones similares en internet.
- 1.2. Estudiar el estado del arte sobre herramientas para la gestión de películas y series para identificar funcionalidades comunes.
- 1.3. Estudiar las tecnologías para el desarrollo de un front-end adaptable (responsivo): React, TypeScript, Tailwind CSS.
- 1.4. Estudiar las tecnologías para el desarrollo del back-end: Java, Spring, servicios REST.
- 1.5. Estudiar herramientas y librerías que puedan ser útiles para el trabajo.

Tarea 2. Análisis y diseño del sistema

Objetivo(s): OB2, OB3

- 2.1. Definir el alcance del sistema.
- 2.2. Establecer los requisitos funcionales y no funcionales del sistema.
- 2.3. Diseñar la base de datos del sistema.
- 2.4. Diseñar el componente back-end del sistema a partir de servicios REST.
- 2.5. Diseñar el componente front-end del sistema siguiendo un modelo responsivo.

Tarea 3. Desarrollo del sistema

Objetivo(s): OB3

- 3.1. Crear el esquema de la base de datos.

- 3.2. Desarrollar los servicios que constituyen la lógica de negocio (back-end).
- 3.3. Desarrollar los componentes que constituyen la interfaz de usuario adaptable (responsive) (front-end).
- 3.4. Conectar el front-end y el back-end.

Tarea 4. Verificación y validación

Objetivo(s): OB3, OB4

- 4.1. Verificar el sistema frente a los requisitos especificados.
- 4.2. Definir los casos de prueba.
- 4.3. Poblar la base de datos con conjunto de datos necesarios para las pruebas.
- 4.4. Validar el sistema a partir de los casos de prueba definidos.
- 4.5. Desplegar el sistema en un entorno de producción.

Tarea 5. Documentación del sistema

Objetivo(s): OB5

- 5.1. Reuniones periódicas con el tutor para revisar el estado del Trabajo Fin de Grado.
- 5.2. Documentar internamente cada uno de los componentes software desarrollados.
- 5.3. Elaborar la memoria del Trabajo Fin de Grado.
- 5.4. Elaborar la presentación del Trabajo Fin de Grado.

En la Tabla 2 se muestra la planificación mensual para las distintas tareas y subtareas a realizar, mientras que en la Tabla 3 se puede ver una estimación de las horas que se van a invertir en cada tarea y subtarea.

Tabla 2. Plan de trabajo: planificación mensual de las tareas

Tarea	10/25	11/25	12/25	01/26	02/26	03/26	04/26	05/26	06/26
T1		X	X						
1.1		X							
1.2		X							
1.3		X	X						
1.4			X						
1.5			X						
T2		X	X	X					
2.1		X							
2.2		X							
2.3		X	X						
2.4		X	X	X					
2.5		X	X	X					
T3			X	X	X	X	X		
3.1			X						
3.2			X	X	X				
3.3				X	X	X			
3.4					X	X	X		
T4				X	X	X	X		
4.1				X					
4.2				X	X				
4.3					X	X			
4.4						X			
4.5							X		
T5		X	X	X	X	X	X	X	

5.1		X	X	X	X	X	X	X	
5.2			X	X	X				
5.3		X	X	X	X	X	X	X	
5.4							X	X	

Tabla 3. Plan de trabajo: estimación de horas por tarea

Tarea	Descripción	Horas estimadas	Horas reales
T1	Análisis preliminar	33	
1.1	Consultar apps similares	2	
1.2	Estudiar estado del arte	4	
1.3	Estudio tecnologías front-end	10	
1.4	Estudio tecnologías back-end	12	
1.5	Estudio herramientas y librerías	5	
T2	Análisis/diseño sistema	40	
2.1	Definir alcance del sistema	2	
2.2	Establecer requisitos	5	
2.3	Diseñar la BBDD	3	
2.4	Diseñar back-end	10	
2.5	Diseñar front-end	20	
T3	Desarrollo de sistema	155	
3.1	Crear esquema de la BBDD	5	
3.2	Desarrollar el back-end	55	
3.3	Desarrollar el front-end	80	
3.4	Conectar back-end y front-end	15	
T4	Verificación/Validación	33	
4.1	Verificar los requisitos	2	
4.2	Definir los casos de prueba	5	
4.3	Poblar la base de datos	10	
4.4	Validar el sistema	13	
4.5	Desplegar el sistema	4	
T5	Documentación	44	
5.1	Reuniones con tutor	6	
5.2	Documentar el desarrollo	8	
5.3	Elaborar la memoria	24	
5.4	Elaborar la presentación	6	
Total		305	YY

4 Diseño y resolución del trabajo realizado

En este capítulo se describe el proceso seguido para el análisis, diseño e implementación de la aplicación MovieMate. El objetivo principal de este apartado es detallar las decisiones técnicas adoptadas, la arquitectura empleada y las soluciones implementadas para dar soporte a las funcionalidades descritas en capítulos anteriores.

4.1 Análisis

En este apartado se describen los **casos de uso** que abarca la aplicación MovieMate, organizados en función del tipo de usuario y de las principales funcionalidades del sistema. El objetivo es ofrecer una visión clara y estructurada de las acciones que puede realizar cada usuario dentro de la plataforma, así como de las restricciones asociadas a cada perfil.

La aplicación distingue entre usuarios no registrados y usuarios registrados. Los usuarios registrados, a su vez, disponen de distintas capacidades relacionadas con la gestión de su perfil, la interacción social y la organización y valoración de contenido audiovisual. Estas funcionalidades se alinean con el objetivo general del proyecto de proporcionar una plataforma unificada para la consulta, gestión y valoración de películas y series, con un marcado componente social.

4.1.1 Usuarios no registrados

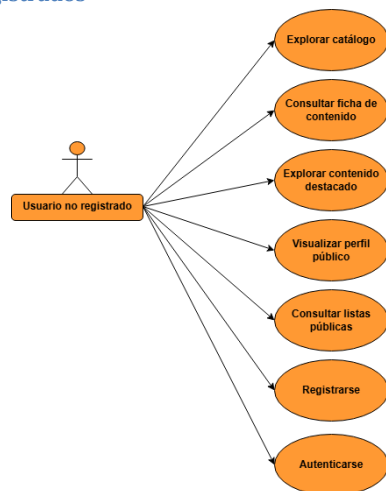


Figura 4. Diagrama de casos de uso de un usuario no registrado

Los usuarios no registrados pueden acceder a una parte limitada de la funcionalidad de MovieMate. Sus principales casos de uso son los siguientes:

- Explorar el catálogo de películas y series.
- Consultar ficha de contenido de películas y series, incluyendo sinopsis y valoraciones agregadas de la comunidad cuando estén disponibles.

Commented [FG7]: Veo recomendable extender la descripción de los CdU utilizando alguna plantilla a tal efecto. El resultado de este ejercicio puede quedar en un anexo al final de la memoria.

Commented [FG8R7]: Pensaba más en una plantilla que incluyese los pasos del CdU y las excepciones (cómo se comportaba el sistema bajo determinadas circunstancias). En cualquier caso, siendo contenido opcional, no creo relevante dedicarle mucho más tiempo.

- Explorar contenido popular o destacado, accediendo a listados generales de películas y series sin necesidad de autenticarse.
- Visualizar perfiles públicos de otros usuarios, incluyendo su nombre, imagen de perfil, biografía y listas públicas, sin posibilidad de interacción directa más allá de la consulta.
- Acceder a listas públicas de contenido creadas por otros usuarios, pudiendo revisar su composición y las valoraciones asociadas al contenido incluido.
- Iniciar el proceso de registro para crear una cuenta nueva.
- Acceder al formulario de inicio de sesión si ya disponen de credenciales

Estas funcionalidades permiten que cualquier visitante pueda hacerse una idea del tipo de contenido y de la experiencia que ofrece MovieMate, al tiempo que se reserva la interacción avanzada y la personalización para los usuarios registrados.

4.1.2 Usuarios registrados

4.1.2.1 Interacción social

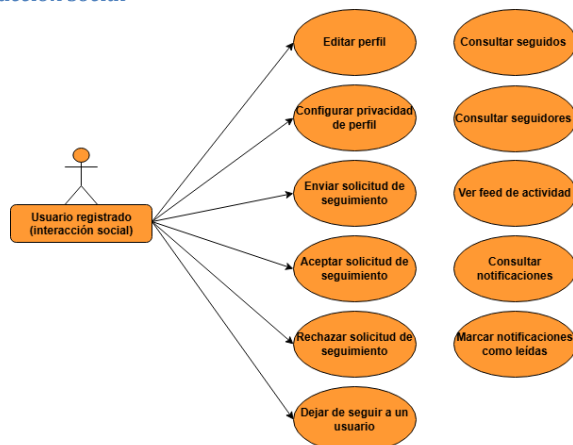


Figura 5. Diagrama de casos de uso de la interacción social de un usuario registrado

Los usuarios registrados disponen de un conjunto ampliado de casos de uso que abarcan desde la gestión de su perfil hasta la interacción social y la organización y valoración de contenido. Entre las funcionalidades generales para usuarios autenticados se incluyen:

- Gestionar su cuenta de usuario, incluyendo la actualización de datos básicos de perfil como nombre, nombre de usuario, dirección de correo electrónico, imagen de perfil y biografía.
- Configurar privacidad del perfil.
- Solicitud de seguimiento a otros usuarios de la plataforma para mantenerse al día de su actividad pública, incluyendo nuevas listas, valoraciones y otras acciones relevantes.
- Aceptar solicitud de seguimiento.
- Rechazar solicitud de seguimiento.
- Dejar de seguir a un usuario cuando se desea dejar de recibir información sobre su actividad.

- Consultar la lista de seguidos, tanto en el propio perfil como en el de otros usuarios cuando la visibilidad lo permita.
- Consultar la lista de seguidores, tanto en el propio perfil como en el de otros usuarios cuando la visibilidad lo permita.
- Acceder a un *feed* de actividad personalizado, donde se muestran acciones recientes de los usuarios seguidos, como nuevas valoraciones, listas creadas o contenido añadido a listas.
- Consultar notificaciones.
- Marcar notificaciones como leídas.

Formatted: Font: Italic

4.1.2.2 Listas personalizadas

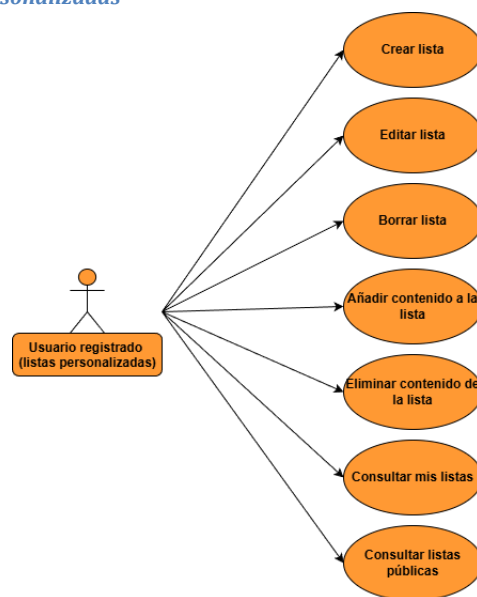


Figura 6. Diagrama de casos de uso sobre listas personalizadas de un usuario registrado

Una de las características centrales de MovieMate es la posibilidad de que los usuarios registrados gestionen listas personalizadas de películas y series, que pueden utilizar tanto a nivel privado como para compartirlas con otros usuarios.

Los casos de uso principales son:

- Crear listas personalizadas, indicando un título, descripción, tipo de lista y nivel de visibilidad (por ejemplo, listas públicas o privadas).
- Editar una lista existente, modificando su nombre, descripción, tipo o parámetros de visibilidad.
- Borrar una lista existente propia.
- Añadir contenido a una lista concreta, seleccionando películas o series previamente consultadas en la aplicación.
- Eliminar contenido de una lista, manteniendo el resto de la lista intacta.

- Consultar mis propias listas.
- Consultar listas públicas de otros usuarios.

Con estas acciones, MovieMate ofrece un mecanismo flexible para organizar contenido según los intereses de cada usuario, como listas de “pendientes por ver”, “favoritas” o colecciones temáticas.

4.1.2.3 Valoraciones y reseñas

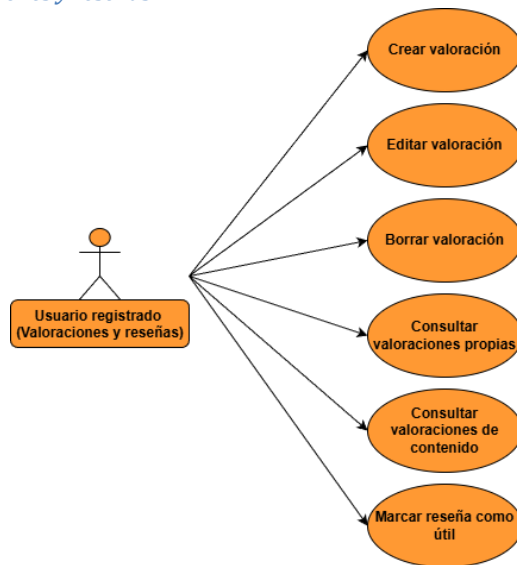


Figura 7. Diagrama de casos de uso de valoraciones y reseñas de un usuario registrado

El sistema de valoraciones y reseñas constituye otro pilar fundamental del funcionamiento de MovieMate, ya que permite a los usuarios expresar su opinión sobre las películas y series que han visto.

Entre los casos de uso asociados a las valoraciones se incluyen:

- Crear una valoración para un contenido determinado, indicando una puntuación y, opcionalmente, un comentario o reseña asociada.
- Editar una valoración, actualizando su puntuación y, opcionalmente, un comentario o reseña asociada.
- Eliminar una valoración previamente registrada.
- Consultar el historial de sus propias valoraciones, accediendo a un listado de películas y series valoradas por el usuario autenticado.
- Consultar las valoraciones asociadas a un contenido concreto, permitiendo ver cómo ha sido recibido por el resto de los usuarios de MovieMate.
- Indicar que una reseña de otro usuario resulta de interés mediante mecanismos de interacción como los “me gusta”.

Estas funcionalidades contribuyen a construir una base de conocimiento colectivo sobre el catálogo, que sirve de apoyo tanto para la exploración de contenido como para el componente social de la plataforma.

Gracias a estas funcionalidades, MovieMate se posiciona no solo como una herramienta de gestión de películas y series, sino también como una red social centrada en la experiencia compartida de descubrir, organizar y valorar contenido audiovisual.

Los casos de uso descritos en este apartado representan el alcance funcional general del sistema y se muestran de forma resumida mediante los diagramas correspondientes. Con el objetivo de proporcionar una especificación más precisa, cada caso de uso principal ha sido documentado utilizando una plantilla formal que describe actores, objetivos y resultados esperados. Esta documentación detallada se incluye en el Anexo A – Especificación de casos de uso.

4.2 Diseño

En este apartado se describe el diseño del sistema desarrollado, comenzando por el modelo de datos sobre el que se apoya la aplicación MovieMate y continuando posteriormente con el diseño de los componentes *back-end* y *front-end* como se muestra en la Figura 8. El modelo de datos constituye la base sobre la que se construyen las funcionalidades de gestión, consulta y valoración de películas y series, así como las características sociales de la plataforma.

Formatted: Font: Italic

Formatted: Font: Italic

El diseño se ha planteado siguiendo una arquitectura en capas, separando claramente la persistencia de datos, la lógica de negocio y la exposición de servicios mediante una API REST, lo que facilita el mantenimiento y la evolución futura del sistema.

4.2.1 Modelo de datos

El modelo de datos constituye la base sobre la que se construyen las funcionalidades de gestión, consulta y valoración de películas y series, así como las características sociales de la plataforma. Como se puede ver en la Figura 8, el modelo de datos de MovieMate se ha diseñado siguiendo un enfoque relacional. El objetivo principal ha sido representar de forma coherente las entidades clave del dominio (películas, series, usuarios, listas, valoraciones y relaciones sociales), así como las relaciones entre ellas, manteniendo un equilibrio entre normalización, eficiencia en las consultas y claridad conceptual.

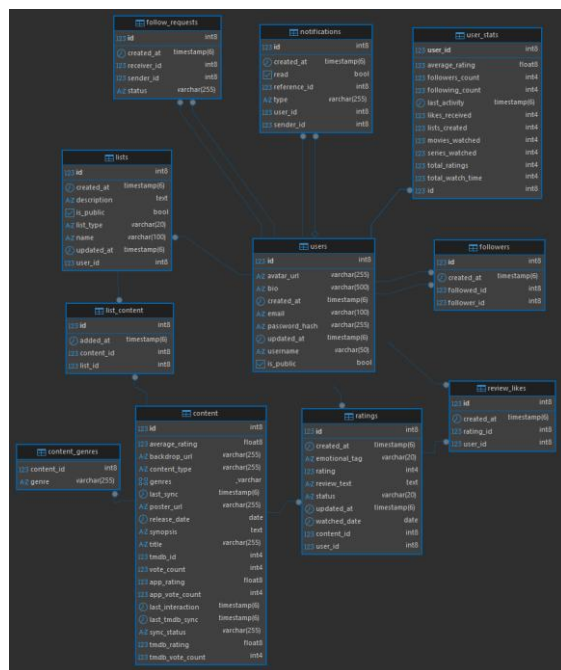


Figura 8. Diagrama entidad-relación de MovieMate

-En el núcleo del modelo se encuentra la entidad **User**, que representa a los usuarios registrados en la plataforma. Cada usuario dispone de un identificador único, credenciales de acceso, información de perfil (nombre de usuario, nombre completo, correo electrónico, biografía e imagen de perfil) y metadatos de auditoría, como las fechas de creación y actualización. Asimismo, la entidad incorpora un atributo que determina el nivel de visibilidad del perfil (público o privado), permitiendo introducir restricciones condicionales en el acceso a la información social del usuario. Esta entidad actúa como punto de anclaje para gran parte de las relaciones del sistema, dado que los usuarios son los responsables de crear listas, emitir valoraciones y establecer vínculos sociales con otros miembros de la plataforma.

-La entidad **Content** modela el contenido audiovisual gestionado por MovieMate, abarcando tanto películas como series mediante un atributo que indica el tipo de contenido. Para cada elemento se almacenan identificadores internos y externos, incluyendo el identificador de TMDb, así como atributos descriptivos como el título, la sinopsis, la fecha de estreno y los recursos gráficos asociados.

Además de los datos descriptivos, la entidad distingue entre métricas externas, procedentes de la API de TMDb (puntuación global y número de votos), y métricas internas generadas por la propia aplicación (puntuación media y número de valoraciones de los usuarios de MovieMate). Esta separación permite mantener independencia respecto a la fuente externa y facilita el cálculo eficiente de estadísticas propias.

Commented [FG9]: Asegúrate de que los nombres de todas las entidades del modelo las pones con la fuente Courier New

Formatted: Font: (Default) Courier New

Formatted: Font: (Default) Courier New

El modelo incorpora también información relacionada con la sincronización del contenido con la API externa. Para ello se almacenan marcas temporales de *‘última actualización’* y *‘última interacción’*, así como un estado que indica si el contenido se encuentra actualizado, potencialmente desactualizado o en proceso de sincronización. Este mecanismo permite adoptar una estrategia de persistencia bajo demanda: el contenido se almacena únicamente cuando existe interacción real por parte de los usuarios, reduciendo así la dependencia directa de la API externa y optimizando el rendimiento del sistema.

En relación con la clasificación temática, los géneros asociados a cada contenido se gestionan mediante una colección almacenada en una tabla independiente, lo que permite modelar de forma flexible la relación entre contenido y categorías sin necesidad de crear una entidad específica para cada género.

La relación entre usuarios y contenido se articula principalmente a través de la entidad *Rating*, que representa las valoraciones y reseñas emitidas por los usuarios sobre películas y series. Cada registro de esta tabla asocia un usuario con un contenido concreto, almacenando la puntuación otorgada, el texto opcional de la reseña y las marcas temporales de creación y actualización. Este diseño permite calcular estadísticas agregadas sobre el contenido, como puntuaciones medias o número de valoraciones, al tiempo que ofrece un historial de la actividad de cada usuario.

La organización personalizada del contenido se materializa mediante la entidad *List*, que recoge las listas creadas por los usuarios para agrupar películas y series según distintos criterios (por ejemplo, “Por ver”, “Favoritas” o colecciones temáticas). Cada lista está asociada a un único usuario propietario y cuenta con atributos como el nombre, la descripción, el tipo de lista y el nivel de visibilidad (pública o privada). La relación muchos-a-muchos entre listas y contenido se resuelve mediante una tabla intermedia que almacena los elementos incluidos en cada lista, permitiendo además incorporar información adicional como el orden de aparición o la fecha de incorporación.

El componente social del sistema se modela a través de la entidad *Follower*, que representa el vínculo entre usuarios seguidores y usuarios seguidos. Cada registro enlaza dos usuarios distintos, evitando duplicidades mediante restricciones de unicidad y previniendo la autorreferencia. Esta estructura permite construir las listas de seguidores y seguidos de cada usuario y soporta funcionalidades como el *feed* de actividad personalizado, basado en las acciones de los usuarios seguidos.

Por otro lado, la entidad *FollowRequest* modela el proceso previo a la confirmación de una relación social cuando el perfil del usuario es privado. Cada solicitud enlaza un usuario emisor y un usuario receptor, incorporando un estado que permite distinguir entre solicitudes pendientes, aceptadas, rechazadas o canceladas. La existencia de esta entidad permite representar explícitamente el ciclo de vida completo de una solicitud de seguimiento y aplicar reglas de negocio diferenciadas en función de su estado.

Con el fin de reforzar la interacción social, el modelo incluye además la entidad *Notification*, destinada a registrar eventos relevantes para los usuarios. Cada notificación se asocia a un usuario receptor y, opcionalmente, a un usuario emisor, almacenando el tipo de

Formatted: Font: Italic

evento que la ha generado y una referencia al recurso relacionado. Entre los eventos contemplados se encuentran la recepción de solicitudes de seguimiento, la aceptación de dichas solicitudes, la incorporación de nuevos seguidores o las interacciones positivas sobre valoraciones. La entidad incorpora asimismo un indicador de lectura que permite distinguir entre notificaciones pendientes y ya procesadas, facilitando la gestión eficiente de la actividad reciente.

Por último, el modelo de datos incluye una entidad `UserStats` destinada a almacenar estadísticas agregadas sobre el comportamiento de los usuarios en la plataforma, tales como el número de películas y series vistas, el total de valoraciones emitidas o el tiempo de visionado acumulado. Esta tabla se encuentra vinculada uno-a-uno con la entidad `User` y se actualiza de forma periódica o bajo demanda a partir de la información registrada en `Ratings` y otros componentes del sistema. Esta separación de responsabilidades permite optimizar las consultas frecuentes sobre estadísticas sin sobrecargar las tablas transaccionales.

En conjunto, el modelo de datos de MovieMate proporciona una base sólida para soportar las funcionalidades centrales del sistema: gestión y valoración de contenido audiovisual, organización mediante listas personalizadas y construcción de una red social alrededor de la actividad de los usuarios. La estructura relacional elegida facilita tanto la consistencia de los datos como la extensibilidad futura del sistema, permitiendo incorporar nuevas entidades y relaciones si se amplían los requisitos funcionales.

Para registrar la interacción de los usuarios sobre las valoraciones, el modelo incorpora dos entidades adicionales. `Comment` almacena los comentarios escritos por otros usuarios sobre una reseña concreta, asociando el texto del comentario tanto al usuario autor como a la valoración referenciada, con soporte de borrado lógico (campo `deleted`). `ReviewLike` modela los «me gusta» que los usuarios pueden dar a las reseñas de otros, garantizando mediante una restricción de unicidad que un mismo usuario no puede dar más de un like a la misma valoración.

El seguimiento del visionado de contenido televisivo se gestiona a través de la entidad `EpisodeWatch`, que almacena los episodios individuales marcados como vistos por cada usuario. Cada registro identifica la serie mediante su identificador de TMDb, el número de temporada y el número de episodio, con una restricción de unicidad sobre la combinación de estos cuatro campos que impide registros duplicados. Los comentarios sobre las listas personalizadas se modelan mediante `ListComment`, que asocia un texto con un usuario autor y la lista comentada, e incorpora igualmente borrado lógico para preservar la integridad referencial.

La plataforma incluye además un sistema de moderación de contenido articulado mediante la entidad `ContentReport`, que permite a los usuarios reportar valoraciones o comentarios inapropiados. Cada reporte registra el tipo de objeto denunciado (`RATING` o `COMMENT`), el motivo de la denuncia (`SPAM`, `INAPPROPRIATE`, `SPOILER` u `OTHER`) y el estado de gestión (`PENDING`, `RESOLVED` o `DISMISSED`). Finalmente, la entidad `UserBadge` materializa el sistema de gamificación de la plataforma, almacenando las insignias concedidas a cada usuario con la fecha de concesión. La combinación de usuario e insignia está sujeta a una restricción de unicidad para garantizar que cada usuario sólo puede obtener cada insignia una vez.

4.2.2 Diseño del back-end

El diseño del *back-end* de MovieMate sigue una arquitectura en tres capas basada en el patrón MVC, donde se separan claramente los controladores web, la lógica de negocio y el acceso a datos. El servidor expone una API RESTful consumida por el cliente web y potencialmente por otros clientes de terceros.

En la capa de presentación se sitúan los controladores REST, encargados de recibir las peticiones HTTP, validar los datos de entrada y devolver respuestas JSON adecuadas. Estos controladores están organizados por dominios funcionales, agrupando *endpoints* relacionados con autenticación, usuarios, contenido, listas, valoraciones, seguidores o actividad. Cada controlador delega la lógica de negocio en servicios especializados, evitando mezclar reglas de negocio con detalles de transporte.

Formatted: Font: Italic

La capa de aplicación está formada por servicios que encapsulan la lógica de negocio de la plataforma, coordinando las operaciones entre las distintas entidades y repositorios. Estos servicios implementan casos de uso como la creación de listas, la emisión de valoraciones, la actualización de estadísticas de usuario o la generación del *feed* de actividad, garantizando la consistencia de los datos y aplicando las validaciones necesarias. Adicionalmente, se incluye un servicio de integración con TMDb, responsable de sincronizar información externa de películas y series con el modelo interno de la aplicación.

Formatted: Font: Italic

La capa de persistencia define repositorios por cada entidad principal del dominio. Estos repositorios proporcionan operaciones CRUD (*Create, Read, Update, Delete*) básicas y consultas derivadas a partir de nombres de método o mediante consultas personalizadas cuando es necesario.

La API REST de MovieMate se ha diseñado siguiendo principios RESTful, usando sustantivos en plural para los recursos y métodos HTTP estándar para expresar las operaciones. A modo de resumen, en la siguiente tabla se describen los principales endpoints disponibles y las operaciones asociadas a cada uno de ellos. Los endpoints que requieren autenticación mediante JWT se indican en la columna correspondiente: los marcados con ✓ requieren que el cliente incluya un token de sesión válido en la cabecera de la petición, mientras que los marcados con - son de acceso público.

Tabla 4. *Endpoints* de la aplicación

Endpoint	GET	POST	PUT	DELETE	Descripción
/api/auth/register		✓			Registrar nuevo usuario
/api/auth/login		✓			Iniciar sesión
/api/users/me/profile			✓		Actualizar perfil de usuario
/api/users/me/public-status			✓		Actualizar visibilidad del perfil
/api/users	✓				Buscar usuarios por <i>query</i>

Commented [FG10]: Convendría distinguir los que están pensados para cualquier usuario frente a los que requieren estar autenticados (se puede utilizar un código de colores para distinguirlos). Si finalmente añades el rol de 'administrador', este también tendría unos endpoints exclusivos para ese rol.

Commented [FG11R10]: NO HECHO

Formatted: Font: Italic

/api/users/{userId}	✓				Obtener un usuario por ID
/api/users/{userId}/stats	✓				Obtener estadísticas de un usuario
/api/users/username/{username}	✓				Obtener un usuario por nombre de usuario
/api/users/suggestions	✓				Obtener sugerencias de usuario
/api/users/me	✓				Obtener usuario actual
/api/users/me/ratings	✓				Obtener valoraciones hechas por el usuario
/api/users/me/lists	✓				Obtener las listas del usuario
/api/users/me/notifications	✓				Obtener las notificaciones del usuario
/api/users/me/follow-requests	✓				Obtener las solicitudes de seguimiento enviadas al usuario
/api/users/{userId}/follow-requests		✓			Mandar solicitud de seguimiento a un usuario
/api/users/follow-requests/{requestId}		✓		✓	Aceptar/rechazar solicitud de seguimiento
/api/users/{userId}/following	✓				Obtener usuarios seguidos por el usuario
/api/users/{userId}/following-status	✓				Comprobar si sigo a un usuario
/api/users/{userId}/followers	✓			✓	Obtener seguidores del usuario
/api/lists		✓			Crear una lista
/api/lists/{listId}/content		✓			Añadir contenido a una lista
/api/lists/public	✓				Obtener todas

					las listas públicas
/api/lists/{listId}/content/{contentId}				✓	Eliminar contenido de una lista
/api/ratings		✓			Crear una valoración
/api/ratings/{ratingId}/likes	✓	✓			Obtener la cantidad de me gustas/Dar me gusta
/api/ratings/{ratingId}/like-status	✓				Comprobar si he dado me gusta a una valoración
/api/ratings/{contentId}	✓				Obtener las valoraciones de un contenido
/api/ratings/{ratingId}				✓	Eliminar una valoración
/api/feed/personal	✓				Obtener el <i>feed</i> personal del usuario
/api/feed/global	✓				Obtener el <i>feed</i> global
/api/notifications/{notificationId}/read			✓		Marcar una notificación como leída
/api/notifications/notifications/read-all			✓		Marcar todas las notificaciones como leídas
/api/content	✓				Obtener todo el contenido
/api/content/{contentId}	✓				Obtener contenido por ID
/api/tmdb/tv/{tmdbId}/sync		✓			Sincronizar serie desde TMDB
/api/tmdb/movies/{tmdbId}/sync		✓			Sincronizar película desde TMDB
/api/tmdb/tv	✓				Buscar series por nombre
/api/tmdb/tv/popular	✓				Buscar series populares
/api/tmdb/movies	✓				Buscar películas por nombre
/api/tmdb/movies/popular	✓				Buscar películas populares

Commented [FG12]: ¿Desde el front le dices al back que haga la sincronización? ¿Quién puede lanzar esa petición?

/api/users/me/recommendations	✓				Obtener recomendaciones personalizadas
/api/users/me/badges	✓				Obtener insignias del usuario actual
/api/users/{userId}/badges	✓				Obtener insignias de un usuario
/api/users/me/avatar		✓			Subir foto de perfil (multipart/form-data)
/api/lists/{listId}			✓	✓	Editar o eliminar una lista propia
/api/tmdb/trending	✓				Obtener contenido en tendencia (películas + series)
/api/ratings/{ratingId}/comments	✓	✓			Obtener o crear comentarios de una valoración
/api/ratings/{ratingId}/comments/{commentId}				✓	Eliminar un comentario de valoración
/api/lists/{listId}/comments	✓	✓			Obtener o crear comentarios de una lista
/api/lists/{listId}/comments/{commentId}				✓	Eliminar un comentario de lista
/api/episodes/watched/summary	✓				Progreso de episodios vistos agrupado por serie
/api/episodes/watched/{tmdbSeriesId}	✓				Obtener episodios vistos de una serie
/api/episodes/watched/{tmdbId}/{season}/{episode}		✓			Marcar o desmarcar un episodio como visto (toggle)
/api/episodes/watched/{tmdbId}/{season}/all		✓		✓	Marcar o desmarcar toda una temporada como vista
/api/reports		✓			Crear un reporte de

					contenido inapropiado
/api/admin/users	✓				Listar todos los usuarios (solo admin)
/api/admin/users/{id}/role			✓		Cambiar el rol de un usuario
/api/admin/users/{id}/ban			✓		Banear o desbanear un usuario
/api/admin/ratings/{id}	✓			✓	Ver o eliminar una valoración (moderación)
/api/admin/comments/{id}	✓			✓	Ver o eliminar un comentario (moderación)
/api/admin/reports	✓				Obtener reportes pendientes de revisión
/api/admin/reports/{id}/resolve			✓		Resolver un reporte (elimina el contenido denunciado)
/api/admin/reports/{id}/dismiss			✓		Desestimar un reporte sin acción

4.2.3 Diseño del front-end

El diseño de la interfaz de usuario de MovieMate se ha elaborado a partir de un conjunto de mockups de alta fidelidad que siguen un sistema visual coherente, basado en una paleta de colores oscura que combina tonos de azul marino profundo y púrpura con un color de acento dorado. Este sistema visual busca evocar la estética propia del cine clásico, ofreciendo una experiencia de usuario diferenciada respecto a las plataformas de referencia.

Los mockups recogen las principales pantallas de la aplicación: la página de inicio con carruseles de contenido destacado y secciones de actividad social, el catálogo de descubrimiento con filtros por género, año y puntuación, la ficha de detalle de una película o serie con información completa e integración de valoraciones, el perfil de usuario con estadísticas y listas, la sección de listas personalizadas y el panel de configuración. Cada pantalla se ha diseñado considerando los casos de uso definidos en el análisis, garantizando que todas las funcionalidades sean accesibles de forma intuitiva.

La estructura de navegación se organiza en torno a una barra lateral fija en escritorio y una barra de navegación inferior en dispositivos móviles, desde las que el usuario accede a las secciones principales: inicio, descubrimiento de contenido, listas, actividad y configuración. El acceso a la ficha de detalle se realiza desde cualquier elemento visual (póster o tarjeta de contenido), manteniendo la coherencia de la navegación.

La aplicación distingue entre rutas públicas, accesibles sin autenticación (página de inicio, catálogo, fichas de contenido y perfiles públicos de usuario), y rutas protegidas, que requieren sesión activa (listas personalizadas, historial de valoraciones, notificaciones y configuración). Esta distinción se aplica tanto en el nivel de navegación como en la presentación de las opciones de interacción disponibles para el usuario.

4.3 Implementación

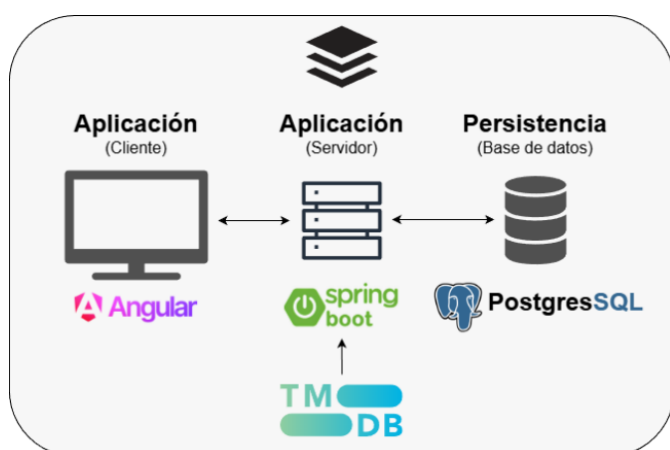


Figura 9. Arquitectura de MovieMate

La arquitectura del sistema constituye uno de los pilares fundamentales en el desarrollo de la aplicación MovieMate, ya que define la forma en la que los distintos componentes del sistema se organizan, interactúan entre sí y evolucionan a lo largo del tiempo. En este proyecto, la arquitectura se ha diseñado con el objetivo de garantizar un sistema robusto, modular, escalable y fácilmente mantenible, alineado con las necesidades funcionales de una aplicación web moderna orientada al consumo intensivo de datos.

MovieMate se concibe como una aplicación web basada en un **modelo cliente-servidor**. En este enfoque, el servidor actúa como proveedor de servicios y concentra la lógica de negocio, la gestión de datos y los mecanismos de seguridad, mientras el cliente se encarga de la interacción con el usuario. Esta separación permite desacoplar responsabilidades, facilitando tanto la evolución independiente de cada componente como la integración con distintos tipos de clientes.

La Figura 9 muestra la arquitectura general del sistema, incluyendo la interacción entre cliente, servidor, base de datos y servicio externo.

Desde el punto de vista arquitectónico, el sistema adopta un enfoque monolítico modular. Esto implica que la aplicación se despliega como una única unidad, pero mantiene una organización interna claramente estructurada en módulos y capas con responsabilidades bien definidas. Este planteamiento favorece la claridad del diseño, simplifica el despliegue y reduce

Formatted: Font: Bold

la complejidad operativa, al tiempo que permite escalar y extender el sistema de forma progresiva.

La elección del enfoque arquitectónico adoptado en MovieMate responde a criterios técnicos orientados a la simplicidad, la mantenibilidad y la capacidad de evolución del sistema.

En primer lugar, se ha optado por una arquitectura basada en **API REST** (*Representational State Transfer*), ya que este estilo arquitectónico permite desacoplar completamente el cliente del servidor. De este modo, el *back-end* puede ser consumido por distintos tipos de clientes (aplicaciones web, móviles o incluso servicios externos) sin necesidad de modificar su implementación interna. Esta decisión resulta especialmente relevante en un proyecto como MovieMate, que aspira a integrar múltiples funcionalidades y a evolucionar con el tiempo.

Formatted: Font: Bold

Formatted: Font: Italic

En segundo lugar, la adopción de una **arquitectura en capas** facilita la separación de responsabilidades dentro del sistema. Cada capa se encarga de un conjunto específico de tareas, como la recepción de peticiones, la aplicación de reglas de negocio o el acceso a datos. Esta organización reduce el acoplamiento entre componentes, mejora la legibilidad del código y favorece la realización de pruebas y tareas de mantenimiento.

Formatted: Font: Bold

La arquitectura incorpora además una integración con la API externa de TMDb para la obtención de información cinematográfica. Esta integración se realiza íntegramente desde el servidor, que actúa como intermediario entre el cliente y la fuente de datos externa. De este modo, las credenciales de acceso a la API permanecen protegidas en el servidor y no se exponen al cliente.

La integración con TMDb se gestiona mediante un servicio dedicado (TmdbService) que encapsula toda la lógica de comunicación con la API externa. Este servicio expone métodos para buscar películas y series, obtener detalles completos de un título, recuperar el reparto, los géneros disponibles o los proveedores de streaming para una región geográfica determinada.

Con el objetivo de reducir la dependencia del servicio externo y optimizar el rendimiento, se ha implementado una estrategia de persistencia bajo demanda. El contenido no se precarga masivamente en la base de datos, sino que se almacena únicamente cuando un usuario interactúa con él (por ejemplo, al valorar una película o añadirla a una lista). Este proceso de sincronización se gestiona a través del método `getOrFetch` del `ContentService`: si el contenido ya existe en la base de datos local, se devuelve directamente; si no existe, se consulta la API de TMDb, se persiste en la base de datos y se devuelve al solicitante. Adicionalmente, cuando el contenido almacenado supera un tiempo de vida (TTL) configurable, se programa una actualización asíncrona en segundo plano para mantener los datos vigentes sin bloquear la respuesta al cliente.

~~La Figura 9 muestra la arquitectura general del sistema, incluyendo la interacción entre cliente, servidor, base de datos y servicio externo.~~

Field Code Changed

~~Asimismo,~~ Se ha descartado la adopción de arquitecturas más complejas, como microservicios, debido a que introducirían una sobrecarga innecesaria en términos de configuración, despliegue y comunicación entre servicios. Para este proyecto, una arquitectura

monolítica bien estructurada ofrece un equilibrio adecuado entre simplicidad y escalabilidad. En la Figura 10, se puede observar de manera visual una comparativa entre la arquitectura monolítica escogida y la arquitectura de microservicios.

En conjunto, estas decisiones arquitectónicas permiten construir un sistema coherente, extensible y alineado con las buenas prácticas del desarrollo de aplicaciones web modernas, garantizando una base sólida para la evolución futura del proyecto.

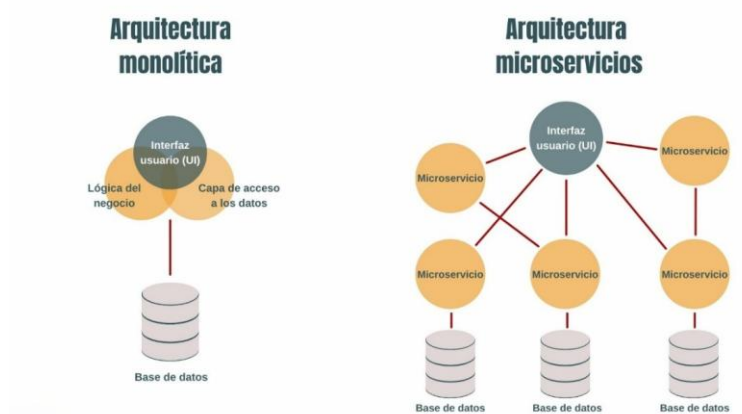


Figura 10. Comparativa entre arquitectura monolítica modular y microservicios [32]

<https://click-it.es/arquitectura-monolitica-vs-arquitectura-de-microservicios-cual-debo-elegir/> Fuente: [32]

4.3.1 Arquitectura del back-end

La arquitectura del *back-end* de MovieMate materializa las decisiones conceptuales descritas en el apartado anterior, implementando una estructura organizada y coherente que garantiza la separación de responsabilidades, la escalabilidad y la mantenibilidad del sistema.

El *back-end* se ha implementado como una aplicación monolítica modular, donde todos los componentes del sistema residen en una única aplicación Spring Boot, pero organizados internamente de forma modular. Este enfoque permite beneficiarse de la simplicidad de despliegue propia de los sistemas monolíticos, sin renunciar a una estructura interna bien definida.

La modularización interna se logra mediante la organización del código en paquetes coherentes, cada uno responsable de un aspecto concreto del sistema, como la gestión de usuarios, el contenido o la seguridad.

Commented [FG13]: Asegúrate de describir en este apartado lo que tenga que ver con la interacción con TMDB (código incluido)

Commented [FG14R13]: Incluye también cualquier fichero de configuración que tengas

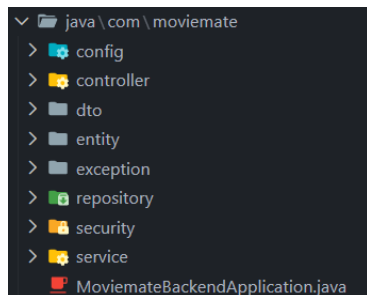


Figura 10. Estructura de carpetas del back-end de MovieMate

El sistema de notificaciones en tiempo real se implementa mediante WebSocket con el protocolo STOMP sobre SockJS. La clase `WebSocketConfig` configura el broker de mensajería y los canales de comunicación, definiendo el prefijo de destino de aplicación (`/app`) y el de usuario (`/user/queue`). La autenticación de las conexiones WebSocket es gestionada por `WebSocketAuthInterceptor`, un interceptor personalizado que extrae el token JWT del frame STOMP CONNECT y lo valida contra el proveedor de autenticación de Spring Security, asignando el principal autenticado a la sesión WebSocket antes de que la suscripción al canal privado del usuario sea efectiva.

Entre los servicios adicionales que complementan la funcionalidad principal del sistema destacan: `BadgeService`, responsable de la evaluación y concesión de los diez tipos de insignias de forma idempotente cada vez que se recalculan las estadísticas de un usuario; `UserStatsService`, que mantiene actualizadas las estadísticas agregadas de cada usuario y genera el `FullStatsDto` empleado por las visualizaciones avanzadas del perfil (distribución de valoraciones, géneros más valorados, actividad mensual); y `CacheCleaner`, un componente planificado (`@Scheduled`) que elimina periódicamente los registros de Content que llevan más de un umbral configurable de días sin interacción, manteniendo la base de datos libre de datos obsoletos poco accedidos.

El back-end incorpora además dos elementos transversales. `SwaggerConfig` configura la documentación automática de la API mediante SpringDoc/OpenAPI, generando una interfaz Swagger UI accesible en `/swagger-ui/index.html` que describe todos los endpoints, sus parámetros, cuerpos de petición y respuestas posibles. Por su parte, `@RequirePublicProfile` es una anotación personalizada que, combinada con `ProfileSecurityService`, permite restringir el acceso a recursos de perfil lanzando `ProfilePrivateException` cuando el perfil objetivo es privado y el solicitante no tiene permisos para consultarlo, simplificando la lógica de autorización en los controladores correspondientes.

4.3.1.1 *Arquitectura en capas*

La arquitectura en capas constituye el eje central sobre el que se estructura el *back-end* de MovieMate. Este patrón arquitectónico permite dividir el sistema en distintos niveles funcionales, cada uno con responsabilidades bien definidas, favoreciendo así la modularidad, la mantenibilidad y la reutilización del código.

En MovieMate, el sistema se organiza en las siguientes capas:

Formatted: Font: Italic

Commented [FG15]: Integra este contenido con el párrafo anterior ('modularización')

- Capa de presentación
- Capa de negocio
- Capa de persistencia
- Capa de seguridad

Estas capas se pueden ver en la **Error! Reference source not found.** Esta estructura está ampliamente utilizada en aplicaciones basadas en Spring Boot. Esta decisión se fundamenta en la necesidad de aislar los distintos aspectos del sistema, evitando acoplamientos innecesarios y facilitando la evolución independiente de cada componente.

4.3.1.1.4.3.2 **Capa de presentación (Controladores)**

La capa de presentación está formada por los controladores REST **(carpeta controller en la Error! Reference source not found.)**, cuya función principal es actuar como punto de entrada de las peticiones HTTP realizadas por los clientes. Estos controladores reciben las solicitudes, validan los datos de entrada y delegan la lógica de negocio en la capa de servicios correspondiente. En la **Error! Reference source not found.** se puede ver un ejemplo de controlador REST de MovieMate.

Una de las decisiones clave en esta capa ha sido mantener los controladores lo más ligeros posible, evitando la inclusión de lógica de negocio compleja. Este enfoque permite reducir la complejidad del código, facilitar la lectura y el mantenimiento, y mejorar la capacidad de prueba aislada de los controladores.

Commented [FG16]: Cada capa debería estar en un encabezado de 3er nivel (4.3.2, 4.3.3, 4.3.4, 4.3.5) integrando todo el contenido que le corresponda (extractos de código y explicación correspondiente). En este sentido, por ejemplo, lo que tiene que ver con la API debería estar en la capa del controlador.

Formatted: Heading 3

Formatted: Font: (Default) Courier New

Commented [FG17]: Haz esto en los distintos apartados para utilizar esa figura como base para articular la organización de esta sección.


```

1  @RestController
2  @RequestMapping("/api/users")
3  @RequiredArgsConstructor
4  public class FollowerController {
5
6      private final FollowerService followerService;
7      private final UserService userService;
8      private final FollowRequestService followRequestService;
9
10     @Operation(
11         summary = "Seguir a un usuario",
12         description = "Permite que el usuario autenticado siga a otro usuario."
13     )
14     @PostMapping("/{userId}/follow-requests")
15     public ResponseEntity<Void> sendFollowRequest(
16         @AuthenticationPrincipal CustomUserDetails userDetails,
17         @Parameter(description = "ID del usuario al que se quiere seguir", example = "1")
18         @PathVariable Long userId) {
19
20         User currentUser = userDetails.getUser();
21         User userToFollow = userService.findUserById(userId);
22         followRequestService.sendFollowRequest(currentUser, userToFollow);
23
24         return ResponseEntity.ok().build();
25     }
26
27     @Operation(
28         summary = "Aceptar solicitud de seguimiento",
29         description = "Permite que el usuario autenticado acepte una solicitud de seguimiento."
30     )
31     @PostMapping("/follow-requests/{requestId}")
32     public ResponseEntity<FollowRequestActionResponse> acceptFollowRequest(@AuthenticationPrincipal CustomUserDetails userDetails, @PathVariable Long requestId) {
33         FollowRequestActionResponse response = followRequestService.acceptRequest(requestId, userDetails.getUser());
34         return ResponseEntity.ok(response);
35     }
36
37     @Operation(
38         summary = "Rechazar solicitud de seguimiento",
39         description = "Permite que el usuario autenticado rechace una solicitud de seguimiento."
40     )
41     @DeleteMapping("/follow-requests/{requestId}")
42     public ResponseEntity<FollowRequestActionResponse> rejectFollowRequest(@AuthenticationPrincipal CustomUserDetails userDetails, @PathVariable Long requestId) {
43         FollowRequestActionResponse response = followRequestService.rejectRequest(requestId, userDetails.getUser());
44         return ResponseEntity.ok(response);
45     }
46 }

```

Figura 11. Ejemplo de controlador REST (FollowerController) en MovieMate

4.3.1.1.24.3.2.1.1 Capa de negocio (Servicios)

La capa de servicios constituye el núcleo funcional del sistema, ya que encapsula toda la lógica de negocio de la aplicación.

En esta capa se implementan, entre otras, las siguientes responsabilidades:

- Gestión de usuarios y autenticación.
- Creación y actualización de valoraciones.
- Organización de contenidos en listas.
- Gestión de relaciones sociales (seguimientos y solicitudes).
- Generación de notificaciones.
- Integración y sincronización con la API externa.

Separar la lógica de negocio en esta capa permite desacoplarla tanto de la presentación como del acceso a datos, favoreciendo la reutilización del código y la posibilidad de modificar la lógica interna sin afectar a los consumidores de la API.

Además, esta capa actúa como intermediaria entre los controladores y los repositorios, coordinando las operaciones necesarias y garantizando la coherencia del sistema. A continuación, en la **Error! Reference source not found.** se puede ver un ejemplo.

Commented [FG18]: Cada figura con código debe contener aspectos que sean representativos de la capa que se está explicando. Es fundamental describir con cierto detalle el contenido de la figura (el código, las distintas etiquetas que aparecen y detalles para que el lector sea capaz de entender lo que ahí pasa).

Commented [FG19R18]: TENER ESTO EN CUENTA PARA TODAS LAS FIGURAS CON CÓDIGO

```

1  @Service
2  @RequiredArgsConstructor
3  public class FollowerService {
4
5      private final FollowerRepository followerRepository;
6      private final FollowRequestRepository followRequestRepository;
7      private final UserService userService;
8
9      @Transactional
10     public void unfollowUser(User followerUser, User userToUnfollow) {
11         Follower follower = followerRepository.findByFollowerAndFollowed(followerUser, userToUnfollow)
12             .orElseThrow(() -> new RuntimeException("No estás siguiendo a este usuario"));
13         followerRepository.delete(follower);
14
15         followRequestRepository
16             .findBySenderAndReceiverAndStatus(
17                 followerUser, userToUnfollow, FollowRequest.FollowRequestStatus.ACCEPTED)
18             .ifPresent(req -> req.setStatus(FollowRequest.FollowRequestStatus.CANCELLED));
19     }
20
21     @Transactional(readOnly = true)
22     public List<UserResponse> getFollowers(Long userId) {
23         User user = userService.findUserById(userId);
24         return followerRepository.findByFollowed(user).stream()
25             .map(f -> userService.mapToUserResponse(f.getFollower()))
26             .collect(Collectors.toList());
27     }
28
29     @Transactional(readOnly = true)
30     public List<UserResponse> getFollowing(Long userId) {
31         User user = userService.findUserById(userId);
32         return followerRepository.findByFollower(user).stream()
33             .map(follower -> follower.getFollowed())
34             .map(userService::mapToUserResponse)
35             .collect(Collectors.toList());
36     }
37
38     @Transactional(readOnly = true)
39     public boolean isFollowing(User follower, User followed) {
40         return followerRepository.existsByFollowerAndFollowed(follower, followed);
41     }
42 }

```

Figura 12. Ejemplo de servicio (FollowerService) en MovieMate

4.3.1.1.34.3.2.1.2 Capa de persistencia (Repositorios)

La capa de persistencia se encarga del acceso a la base de datos PostgreSQL y se implementa mediante Spring Data JPA como se puede observar en la **Error! Reference source not found..** El uso de JPA (*Java Persistence API*) permite mapear las entidades Java al modelo relacional de PostgreSQL, aprovechando características como las relaciones entre tablas, la gestión de transacciones y la paginación nativa en las consultas.

Los repositorios actúan como interfaces de acceso a datos, proporcionando operaciones CRUD (*Create, Read, Update, Delete*) y consultas personalizadas cuando es necesario. Esta abstracción evita la inclusión de consultas SQL explícitas en la lógica de negocio y mejora la portabilidad del sistema frente a posibles cambios del gestor de base de datos.

Durante el diseño se ha priorizado el uso de carga diferida (*lazy loading*) en las relaciones entre entidades, con el objetivo de evitar recuperaciones innecesarias de información y optimizar el rendimiento.

Este enfoque ofrece varias ventajas:

- Reducción del código repetitivo.
- Mayor legibilidad.
- Portabilidad frente a cambios en el sistema gestor de bases de datos.

```
1 public interface FollowerRepository extends JpaRepository<Follower, Long> {
2     Optional<Follower> findByFollowerAndFollowed(User follower, User followed);
3     List<Follower> findByFollower(User follower); // Usuarios que yo sigo
4     List<Follower> findByFollowed(User followed); // Mis seguidores
5     boolean existsByFollowerAndFollowed(User follower, User followed);
6
7     @Query("SELECT COUNT(f) FROM Follower f WHERE f.followed.id = :userId")
8     Integer countFollowersByUserId(@Param("userId") Long userId);
9
10    @Query("SELECT COUNT(f) FROM Follower f WHERE f.follower.id = :userId")
11    Integer countFollowingByUserId(@Param("userId") Long userId);
12
13    Page<Follower> findByFollowerInAndCreatedAtAfterOrderByCreatedAtDesc(
14        List<User> followers,
15        LocalDateTime createdAt,
16        Pageable pageable
17    );
18
19    @Query("SELECT COUNT(f) FROM Follower f WHERE f.followed = :user")
20    Integer countByFollowed(@Param("user") User user);
21
22    @Query("SELECT COUNT(f) FROM Follower f WHERE f.follower = :user")
23    Integer countByFollower(@Param("user") User user);
24 }
```

Figura 13. Ejemplo de repositorio JPA (FollowerRepository) en MovieMate

4.3.1.1.4.3.2.1.3 Capa de seguridad

La seguridad se implementa como una capa transversal apoyada en Spring Security. Esta capa intercepta las peticiones entrantes antes de que alcancen los controladores, aplicando mecanismos de autenticación y autorización, como se puede observar en la Figura 14.

El sistema adopta un enfoque sin estado (*stateless*) basado en JSON Web Tokens (JWT). Tras validar las credenciales del usuario, el servidor genera un token firmado digitalmente que el cliente debe incluir en las peticiones posteriores. En cada solicitud, un filtro de seguridad verifica la validez del token y construye un contexto de seguridad asociado al usuario autenticado.

A partir de dicho contexto, se aplican reglas de autorización que restringen el acceso a determinados endpoints según el estado de autenticación o los roles asignados.

Esta separación permite mantener la lógica de seguridad desacoplada de la lógica de negocio, reforzando el principio de responsabilidad única.

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(postFilterEnabled = true)
@RequestScope
public class SecurityConfig {

    private final UserRepository userRepository;

    @Bean
    public UserDetailsServiceImpl userDetailsService() {
        return username -> userRepository.findByUsernameOrEmail(username, username)
            .map(UserDetails::new)
            .orElseThrow(() -> new UsernameNotFoundException("Usuario no encontrado"));
    }

    @Bean
    public AuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(userDetailsService());
        provider.setPasswordEncoder(passwordEncoder());
        return provider;
    }

    @Bean
    public JwtAuthenticationFilter jwtAuthenticationFilter(JwtService jwtService, UserDetailsService userDetailsService) {
        return new JwtAuthenticationFilter(jwtService, userDetailsService);
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http, JwtAuthenticationFilter jwtAuthenticationFilter) throws Exception {
        http
            .csrf().disable()
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/swagger-ui.html",
                    "/swagger-ui/**",
                    "/api-docs/**",
                    "/api-docs/**",
                    "/swagger-resources/**",
                    "/webjars/**",
                    "/actuator/**",
                    "/api/auth/**",
                    "/api/content/**",
                    "/api/files/public/**",
                    "/api/users/**",
                    "/api/users/{userId:[0-9]+}",
                    "/api/users/{userId:[0-9]+}/profile/**",
                    "/api/users/{username}/**",
                    "/api/ratings/{ratingId:[0-9]+}/likes/**",
                    "/api/ratings/**",
                    "/api/notifications/**")
                .permitAll()
                .anyRequest().authenticated()
            )
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            .authenticationProvider(authenticationProvider())
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

Figura 14. Configuración de seguridad

4.3.1.24.3.2.2 Diseño de la API REST

La API REST constituye el principal mecanismo de interacción entre el cliente y el back-end. Su diseño sigue los principios del estilo arquitectónico REST (*Representational State Transfer*), adoptando un enfoque orientado a recursos.

Cada entidad relevante del dominio (usuarios, contenidos, valoraciones, listas, relaciones sociales, notificaciones, etc.) se expone como un recurso accesible mediante una URI específica. Las operaciones sobre dichos recursos se realizan utilizando los métodos HTTP estándar (GET, POST, PUT, DELETE), garantizando una semántica clara y coherente como se puede ver en la Figura 15.

follower-controller			
GET	/api/users/{userId}/followers	Obtener seguidores de un usuario	🔒
POST	/api/users/{userId}/followers	Seguir a un usuario	🔒
DELETE	/api/users/{userId}/followers	Dejar de seguir a un usuario	🔒
GET	/api/users/{userId}/following	Obtener usuarios seguidos por un usuario	🔒
GET	/api/users/{userId}/following-status	Comprobar si el usuario autenticado sigue a otro usuario	🔒

Figura 15. Endpoints para gestión de seguidores en MovieMate

Asimismo, la API emplea de forma sistemática códigos de estado HTTP para reflejar el resultado de cada operación, tales como:

- **200 OK** para operaciones exitosas.
- **201 Created** para la creación de nuevos recursos.
- **400 Bad Request** para peticiones mal formadas.
- **401 Unauthorized** para accesos no autenticados.
- **403 Forbidden** para accesos sin permisos suficientes.
- **404 Not Found** para recursos inexistentes.

El uso consistente de estas convenciones facilita la integración con distintos clientes y mejora la interpretabilidad de las respuestas.

4.3.1.2.14.3.2.2.1 Uso de DTOs en la arquitectura

La arquitectura de la API se apoya en el uso de *Data Transfer Objects* (DTOs), que actúan como intermediarios entre el modelo interno del sistema y la interfaz pública.

Los DTOs permiten:

- Controlar explícitamente la información expuesta al cliente.
- Evitar la exposición directa de entidades JPA.
- Reducir el acoplamiento entre la capa de persistencia y la API.
- Facilitar la evolución del modelo interno sin afectar a los consumidores externos.

Esta decisión resulta especialmente relevante en un sistema con múltiples relaciones entre entidades y atributos sensibles asociados a los usuarios.

En la Figura 16 se puede ver un ejemplo de un DTO de respuesta a una petición de contenido, ya sea serie o película.



```

1  @Data
2  @Builder
3  public class ContentResponse {
4      private Long id;
5      private Integer imdbId;
6      private String title;
7      private Content.ContentType contentType;
8      private String releaseDate;
9      private String posterUrl;
10     private String backdropUrl;
11     private String synopsis;
12     private String[] genres;
13     private Double averageRating;
14     private Integer voteCount;
15     private String lastSync;
16 }

```

Figura 16. Ejemplo DTO

4.3.1.34.3.2.3 Flujo de una petición

El procesamiento de una petición en el back-end sigue una secuencia bien definida. En primer lugar, el cliente envía una petición HTTP a la API REST, que es interceptada por los filtros de seguridad para validar el token JWT. A continuación, el controlador correspondiente recibe la petición, valida los datos de entrada y delega la lógica de negocio en el servicio asociado. El servicio aplica las reglas de negocio, interactúa con los repositorios para acceder o modificar los datos en la base de datos, y devuelve el resultado al controlador. Finalmente, el controlador transforma el resultado en un DTO y lo devuelve al cliente en forma de respuesta JSON con el código de estado HTTP adecuado.

Este flujo garantiza una clara separación de responsabilidades

4.3.24.3.3 Arquitectura del front-end

El front-end de MovieMate se ha implementado como una Single Page Application (SPA) desarrollada con React 19 y TypeScript. La arquitectura sigue un enfoque modular centrado en características (feature-based), organizando el código en módulos independientes que agrupan páginas, componentes, hooks y lógica de acceso a datos. A continuación se describen los principales aspectos estructurales y las páginas que conforman la aplicación.

4.3.24.3.3.1 Estructura del proyecto

La raíz del código fuente se organiza en los siguientes directorios principales dentro de src/:

- pages/ — vistas completas de la aplicación, cada una asociada a una ruta navegable.
- features/ — módulos por funcionalidad (listas, valoraciones, perfil, notificaciones, etc.), cada uno con sus propios componentes, hooks y lógica.
- components/ — componentes compartidos y reutilizables (tarjetas de póster, modales, botones de acción) empleados en múltiples páginas.
- hooks/ — hooks personalizados que encapsulan lógica de acceso a datos y gestión de estado.
- api/ — módulos de acceso a la API del servidor organizados por recurso: usersApi, listsApi, ratingsApi, contentApi, notificationsApi.

- stores/ — estado global sincrónico gestionado con Zustand (tienda de autenticación useAuthStore).
- lib/ — utilidades y funciones auxiliares: cliente Axios preconfigurado, jerarquía de claves de caché (queryKeys), helpers de slugificación y formato.
- types/ — definiciones de tipos TypeScript compartidas entre módulos.

Esta organización favorece la cohesión interna de cada módulo, reduce el acoplamiento entre características y facilita la localización del código durante el mantenimiento y la evolución de la aplicación.

[Figura 4.1: Árbol de directorios del proyecto front-end de MovieMate.]

4.3.2.24.3.3.2 Navegación y autenticación

La navegación entre páginas se gestiona mediante React Router v6. Las rutas se declaran de forma centralizada en el componente raíz App.tsx, distinguiendo entre rutas públicas (accesibles sin sesión) y rutas protegidas (que requieren autenticación). El acceso a estas últimas se controla mediante el componente guardián PrivateRoute, que comprueba el estado de autenticación almacenado en Zustand y redirige al usuario a la pantalla de inicio de sesión si no dispone de sesión activa.

El flujo de autenticación sigue los pasos siguientes: el usuario introduce sus credenciales en el formulario; la petición se envía al endpoint POST /api/auth/login; si las credenciales son válidas, el servidor devuelve un token JWT que se almacena tanto en el store de Zustand (estado en memoria) como en el almacenamiento local del navegador (persistencia entre recargas de página); a partir de ese momento, una instancia global de Axios incorpora automáticamente el token como cabecera Authorization: Bearer <token> en cada petición autenticada.

[Figura 4.2: Diagrama del flujo de autenticación y protección de rutas en el front-end.]

4.3.2.24.3.3.3 Gestión del estado y comunicación con el servidor

La gestión del estado se divide en dos capas claramente separadas para evitar mezclar lógica de servidor y estado local en los componentes.

El estado global sincrónico, como los datos del usuario autenticado y el token JWT, se gestiona mediante Zustand. Su store reactivo de bajo peso permite que cualquier componente suscriba cambios de forma eficiente sin renderizados innecesarios.

El estado asíncrono derivado del servidor se gestiona con TanStack Query v5, que automatiza el ciclo de vida de las peticiones HTTP: almacena los resultados en caché mediante un sistema de claves estructuradas (queryKeys), ejecuta revalidaciones transparentes al recuperar el foco de la ventana o superar el tiempo de stale, y expone hooks de mutación (useMutation) que invalidan selectivamente las entradas de caché afectadas al confirmar un cambio. Por ejemplo, al añadir contenido a una lista se invalidan simultáneamente las claves lists.mine() y users.lists(), garantizando que todos los componentes suscritos reflejen el cambio de inmediato.

Todas las comunicaciones HTTP se canalizan a través de módulos de API dedicados que emplean la instancia global de Axios preconfigurada con el interceptor de autenticación y la URL base del servidor.

[Figura 4.3: Diagrama de las capas de gestión del estado en el front-end: Zustand para estado sincrónico y TanStack Query para estado del servidor.]

4.3.2.44.3.3.4 *Página de inicio*

La página de inicio (HomePage, ruta /) es el punto de entrada principal de la aplicación. Para usuarios no autenticados muestra un bloque de bienvenida con una llamada a la acción que invita al registro. Para usuarios autenticados ofrece tres secciones de contenido personalizado:

- Contenido en tendencia: carrusel horizontal alimentado por el endpoint `GET /api/tmdb/trending`, que muestra los títulos cinematográficos y televisivos más populares del momento.
- Recomendaciones personalizadas («Para ti 🌟»): carrusel generado a partir del historial de valoraciones del usuario mediante `GET /api/users/me/recommendations`. El algoritmo selecciona los géneros más frecuentes en las valoraciones altas del usuario y recupera títulos relacionados que aún no han sido valorados.
- Usuarios sugeridos: lista de perfiles cinéfilos propuestos para seguir, fomentando la dimensión social de la plataforma.

[Figura 4.4: Captura de la página de inicio con las secciones de tendencias, recomendaciones personalizadas y usuarios sugeridos.]

4.3.2.54.3.3.5 *Página de descubrimiento*

La página de descubrimiento (DiscoverPage, ruta /discover) centraliza la función de búsqueda de la aplicación. Dispone de una barra de búsqueda con debounce que lanza consultas simultáneas a la API de películas y de series, interleaving los resultados para ofrecer una experiencia de búsqueda mixta sin necesidad de seleccionar un tipo de contenido. Cuando la barra está vacía se muestran los contenidos en tendencia como estado por defecto.

Los resultados se presentan en una cuadrícula de tarjetas (ContentCard) que muestran el póster, el título y el año de estreno. Cada tarjeta es un enlace a la ficha de detalle del contenido. La cuadrícula utiliza flex flex-wrap para adaptarse a cualquier resolución de pantalla sin dejar celdas vacías.

[Figura 4.5: Captura de la página de descubrimiento con resultados de búsqueda mixtos de películas y series.]

4.3.2.64.3.3.6 *Ficha de detalle de contenido*

La ficha de detalle (ContentDetailPage, ruta /content/:contentType/:tmdbId/:slug) es la vista más compleja de la aplicación y el núcleo de la experiencia de usuario. Recibe el tipo de contenido (MOVIE o TV) y el identificador de TMDB como parámetros de ruta. La página se estructura en los siguientes bloques:

- Cabecera (ContentHeader): muestra el backdrop a pantalla completa con degradado superpuesto, el póster a la izquierda y los metadatos principales a la derecha (título, año, géneros, puntuación TMDb, número de votos).
- Widget de valoración (RatingWidget): permite al usuario puntuar el contenido con 1 a 5 estrellas y adjuntar una reseña textual. Si el usuario ya realizó una valoración previa, la carga automáticamente para permitir su edición.
- Gestión de listas: botón que abre el diálogo AddContentToListDialog, desde el que el usuario puede añadir el contenido a cualquiera de sus listas (Watchlist, Favoritos, Visto o listas personalizadas).
- Reparto y equipo: galería de tarjetas PersonCard con los principales integrantes, cada una enlazada a la ficha de la persona (/person/:personId/:slug).
- Seguimiento de episodios (solo series TV): acordeón SeasonAccordion que despliega las temporadas y permite marcar episodios individuales como vistos, registrando la acción en la entidad EpisodeWatch.

[Figura 4.6: Captura de la ficha de detalle de una película mostrando el widget de valoración, los metadatos principales y el carrusel de reparto.]

4.3.2.7 4.3.3.7 Perfil de usuario

La página de perfil (ProfilePage, ruta /profile/:username) muestra la información pública y la actividad de un usuario. Soporta deep-linking mediante el parámetro de consulta ?tab= para anclar directamente una pestaña concreta. El perfil se organiza en cuatro pestañas:

- Actividad: línea de tiempo cronológica con las valoraciones, reseñas y movimientos en listas del usuario, con paginación cliente en bloques de cinco elementos.
- Listas: cuadrícula de las listas del usuario con acceso directo a cada ListDetailPage.
- Estadísticas (StatsTab): visualizaciones avanzadas generadas a partir del endpoint GET /api/users/:id/stats, que devuelve un FullStatsDto con la distribución de valoraciones (1-5), los géneros más valorados y la actividad mensual.
- Insignias (BadgesSection): colección de insignias obtenidas por el usuario a través del uso de la plataforma, gestionadas por BadgeService con diez tipos definidos (primera valoración, primera lista, diez valoraciones, etc.).

Para perfiles privados o usuarios no autenticados la página gestiona el código HTTP 403 (ProfilePrivateException) mostrando un mensaje de perfil privado sin reintentar la petición. Los diálogos FollowListDialog muestran la lista de seguidores y de cuentas seguidas. El componente QuickEditRatingDialog permite editar una valoración directamente desde la cuadrícula de pósters sin abandonar la página.

[Figura 4.7: Captura del perfil de usuario mostrando la pestaña de estadísticas avanzadas con gráficos de distribución de valoraciones y géneros.]

4.3.2.8 4.3.3.8 Listas personalizadas

La funcionalidad de listas personalizadas se articula en tres páginas complementarias que comparten la capa de acceso a datos pero difieren en su presentación y propósito.

ListsPage (/lists) muestra la cuadrícula de listas del usuario autenticado. Cada lista se representa mediante el componente MyListCard, que ofrece acceso directo a las acciones de edición (EditListDialog, para modificar nombre, descripción y visibilidad) y eliminación. Desde esta misma página se puede crear una nueva lista personalizada con nombre único por usuario.

ListDetailPage (/lists/:listId) muestra el contenido completo de una lista concreta con carga instantánea gracias al patrón placeholderData, que recupera los datos preliminares del estado de navegación (location.state) mientras se completa la petición al servidor. En la parte inferior aparece la sección de comentarios, donde cualquier usuario puede dejar valoraciones públicas sobre la lista (entidad ListComment, gestionada por ListCommentService).

SpecialListPage es una vista compartida para las rutas /watchlist, /favorites y /watched, diferenciadas únicamente por el prop listType (ListType.WATCHLIST, FAVORITES, WATCHED). El diálogo AddContentToListDialog permite buscar películas y series directamente desde la interfaz de la lista, deduplicando los resultados para evitar sugerir contenido ya presente en la misma.

[Figura 4.8: Captura de la página de detalle de una lista personalizada con la cuadrícula de pósters y la sección de comentarios.]

4.3.2.94.3.3.9 *Actividad, notificaciones y comunicación en tiempo real*

La página de actividad (ActivityPage, ruta /activity) muestra un feed cronológico de las acciones recientes de los usuarios seguidos: nuevas valoraciones, reseñas publicadas o modificadas, y añadidos a listas. El feed se obtiene del endpoint GET /api/activity y se presenta con paginación. ActivityService distingue entre valoraciones nuevas y actualizadas comparando updatedAt con createdAt para generar los eventos RATING_UPDATED y LIST_UPDATED de forma precisa.

La página de notificaciones (NotificationsPage, ruta /notifications) lista los eventos dirigidos al usuario autenticado. El recuento de notificaciones no leídas se refleja en un badge sobre el icono de la barra de navegación.

Para la recepción de notificaciones en tiempo real, el componente raíz Layout.tsx inicializa al cargar la aplicación una conexión WebSocket mediante SockJS y el protocolo STOMP. El token JWT se incluye en las cabeceras del frame STOMP CONNECT y es validado por WebSocketAuthInterceptor en el servidor. Cuando se emite un evento, el cliente actualiza la caché de TanStack Query con setQueryData para añadir la nueva notificación de forma reactiva e inmediata, sin necesidad de una nueva petición HTTP, e invalida la consulta para mantener la coherencia a largo plazo.

[Figura 4.9: Captura de la bandeja de notificaciones de MovieMate con el indicador de notificaciones pendientes.]

4.3.2.104.3.3.10 *Sistema de diseño y adaptabilidad*

La interfaz de MovieMate se construye sobre un sistema de diseño coherente definido mediante tokens de Tailwind CSS 4. Los principales tokens son:

- Paleta oscura estructurada: clases bg-bg-0 a bg-bg-3 para los distintos niveles de profundidad de los fondos, con gradaciones del azul marino profundo al gris carbón.
- Acento dorado (#e8c97a): empleado como color primario de interacción en botones de acción, estrellas de valoración, iconos activos y elementos de énfasis.
- Tipografía dual: Playfair Display para encabezados y títulos de películas, DM Sans para el cuerpo del texto y controles, DM Mono para fechas y datos numéricos.

Los componentes accesibles de Radix UI y Shadcn/ui garantizan la compatibilidad con lectores de pantalla y el cumplimiento de los estándares WCAG. La interfaz es completamente adaptable (responsive): en pantallas móviles aparece una barra de navegación inferior fija (BottomNavBar, visible solo en resoluciones inferiores a lg), mientras que en escritorio se emplea una barra de navegación lateral. Esta adaptabilidad se logra exclusivamente con Tailwind CSS sin necesidad de librerías de layout adicionales.

[Figura 4.10: Comparativa de la interfaz de MovieMate en escritorio y dispositivo móvil mostrando la barra de navegación adaptativa.]

4.4 Pruebas

La calidad del software desarrollado en MovieMate se ha reforzado mediante la incorporación de un conjunto de pruebas automatizadas orientadas principalmente a la validación de la lógica de negocio del sistema.

La estrategia adoptada se centra en pruebas unitarias sobre la capa de servicios, ya que esta constituye el núcleo funcional del *back-end*. Probar esta capa de forma aislada permite verificar que las reglas de negocio se aplican correctamente sin depender del acceso real a la base de datos ni de otros componentes externos.

Para la implementación de las pruebas se ha utilizado **JUnit 5** como *framework* de *testing* y **Mockito** como herramienta de simulación (*mocking*) de dependencias. Este enfoque permite sustituir los repositorios reales por objetos simulados, garantizando que las pruebas evalúan exclusivamente el comportamiento del servicio bajo condiciones controladas.

Un ejemplo representativo es la clase de pruebas asociada al servicio de gestión de listas. En ella se validan distintos escenarios relacionados con la creación de listas personalizadas, la adición y eliminación de contenido, así como el control de permisos de acceso.

Entre los casos cubiertos *se incluyen*:

- Creación correcta de una lista cuando el nombre es único.
- Detección de nombres duplicados.
- Restricción de listas de tipo único por usuario.
- Control de acceso cuando un usuario intenta modificar una lista que no le pertenece.
- Gestión de errores cuando el contenido no existe.
- Prevención de inserciones duplicadas en una lista.

Asimismo, se comprueba el correcto mapeo de entidades a objetos de transferencia (DTOs), garantizando que la información devuelta al cliente cumple con el contrato definido por la API.

Commented [FG20]: TODOS LOS TÉRMINOS EN INGLÉS EN CURSIVA

Formatted: Font: Italic

Formatted: Font: Bold

Formatted: Font: Italic

Formatted: Font: Bold

Commented [FG21]: Puede estar bien poner aquí extractos de código con este mismo ejemplo que estas dando.

• Este enfoque aporta múltiples beneficios:

- Permite detectar errores en etapas tempranas del desarrollo.
- Reduce el riesgo de regresiones al introducir nuevas funcionalidades.
- Facilita la refactorización del código al disponer de una red de seguridad automatizada.
- Mejora la mantenibilidad del sistema.

Formatted: Normal, No bullets or numbering

Commented [FG22]: Hazlo párrafo

Las pruebas se ejecutan automáticamente durante el proceso de integración continua, asegurando que cualquier modificación en el código sea validada antes de su incorporación definitiva al repositorio principal.

En conjunto, la adopción de pruebas unitarias automatizadas contribuye significativamente a la robustez y fiabilidad del sistema, alineándose con buenas prácticas de desarrollo profesional.

4.5 Despliegue

Con el objetivo de garantizar la portabilidad, reproducibilidad y automatización del entorno de ejecución, el despliegue del *back-end* de MovieMate se ha basado en tecnologías de contenerización y en la integración continua mediante *pipelines* automatizados.

Formatted: Font: Italic

Esta estrategia permite:

- Asegurar coherencia entre entornos de desarrollo y producción.
- Simplificar la configuración de dependencias.
- Automatizar la validación y construcción del proyecto.
- Facilitar la escalabilidad futura del sistema.

Commented [FG23]: Evita estas listas en la medida de lo posible; en su lugar, escríbelo en forma de párrafo.

4.5.1 Contenerización con Docker

Para encapsular la aplicación back-end se ha utilizado Docker. La aplicación se construye mediante un proceso *multi-stage* definido en el Dockerfile, lo que permite separar la fase de compilación de la fase de ejecución.

En la primera etapa (*build stage*):

- Se emplea una imagen basada en Maven con Java 21.
- Se compila el proyecto mediante el comando `mvn clean package`.
- Se genera el archivo JAR ejecutable.

En la segunda etapa (*runtime stage*):

- Se utiliza una imagen ligera basada en Eclipse Temurin JRE 21.
- Se crea un usuario no privilegiado para mejorar la seguridad.
- Se copia únicamente el JAR generado en la etapa anterior.
- Se expone el puerto 8080.
- Se configura la ejecución mediante variables de entorno (`JAVA_OPTS`).

El uso de *multi-stage builds* reduce significativamente el tamaño final de la imagen, ya que elimina dependencias innecesarias del entorno de compilación.

Esta aproximación aporta:

- Mayor seguridad.
- Imágenes más ligeras.
- Entornos reproducibles.
- Aislamiento del sistema anfitrión.

4.5.2 Orquestación con Docker Compose

Para facilitar el despliegue conjunto del *back-end* y la base de datos se ha utilizado Docker Compose.

Se han definido dos servicios principales:

Base de datos PostgreSQL

- Imagen oficial PostgreSQL 16 (versión Alpine).
- Persistencia mediante volumen Docker.
- Configuración de credenciales a través de variables de entorno.
- *Healthcheck* para verificar disponibilidad antes de iniciar el *back-end*.

Back-end de la aplicación

- Construido a partir del Dockerfile definido.
- Dependencia explícita del servicio de base de datos.
- Configuración completa mediante variables de entorno:
 - Conexión a base de datos.
 - Configuración JWT.
 - Parámetros de integración con TMDB.
 - Orígenes permitidos para CORS.
 - Perfil activo de Spring (prod).

La utilización de Docker Compose permite levantar todo el entorno con un único comando, simplificando la puesta en marcha del sistema y reduciendo errores de configuración manual.

Además, la red interna definida garantiza la comunicación segura entre contenedores sin exponer servicios innecesarios al exterior.

4.5.3 Integración continua y automatización (CI/CD)

Con el objetivo de asegurar la calidad del código y automatizar el ciclo de construcción del proyecto, se ha implementado un *pipeline* de integración continua mediante **GitHub Actions**.

El flujo definido se ejecuta automáticamente cuando

- Se realizan *pushes* a las ramas *main* o *develop*.
- Se abre una *pull request* hacia la rama principal.

El pipeline se compone de los siguientes trabajos:

Ejecución de pruebas

Commented [FG24]: Muéstralo en la parte de implementación del backend

Formatted: Font: Italic

Formatted: Font: Bold

- Configuración automática de Java 21.
- Ejecución de pruebas unitarias con Maven.
- Generación opcional de informe de cobertura (JaCoCo).

Este paso garantiza que cualquier modificación en el código sea validada antes de su integración definitiva.

Construcción del artefacto

- Compilación del proyecto.
- Generación del archivo JAR.
- Publicación temporal del artefacto como resultado del *workflow*.

Verificación de calidad de código

- Ejecución de reglas de estilo mediante *Checkstyle*.
- Detección temprana de posibles problemas de calidad.

Construcción de imagen Docker

En caso de *push* a la rama principal:

- Se construye automáticamente la imagen Docker del *back-end*.
- Se optimiza el proceso mediante cacheo de capas.
- Se prepara la imagen para un posible despliegue en producción.

Estrategia de ramificación

El proyecto sigue una estrategia de ramificación (branching strategy) basada en dos ramas permanentes. La rama *develop* concentra todo el desarrollo activo: cada nueva funcionalidad, corrección de errores o tarea de refactorización se implementa en esta rama antes de ser integrada. Sobre cada *push* a *develop* se ejecuta automáticamente el pipeline de CI/CD, que valida el código mediante pruebas unitarias y análisis estático antes de construir el artefacto.

La rama *main* representa el estado estable y desplegable de la aplicación. Un *merge* desde *develop* a *main* únicamente se realiza cuando el pipeline completo ha concluido satisfactoriamente, garantizando que la rama principal nunca contiene código en estado incorrecto. Un *merge* exitoso a *main* desencadena adicionalmente el paso de construcción y publicación de la imagen Docker, preparando la nueva versión para su despliegue.

Esta separación entre rama de integración (*develop*) y rama de producción (*main*) proporciona un cortafuegos de calidad automatizado, elimina la dependencia de revisiones manuales para detectar regresiones y facilita la trazabilidad entre versiones desplegadas y commits específicos de la rama principal.

4.5.4 Despliegue en Kubernetes con Minikube

Kubernetes es una plataforma de orquestación de contenedores de código abierto que automatiza el despliegue, el escalado y la gestión de aplicaciones contenerizadas. Permite describir el estado deseado del sistema mediante ficheros de configuración declarativos

(manifestos YAML) y se encarga de reconciliar continuamente el estado real del clúster con el estado deseado.

Para facilitar la validación local del despliegue orquestado sin necesidad de infraestructura en la nube, se ha utilizado Minikube. Esta herramienta levanta un clúster Kubernetes de un único nodo directamente en el equipo del desarrollador, simulando un entorno de producción real con un coste operativo mínimo.

El despliegue de MovieMate en Kubernetes se describe mediante los siguientes recursos:

- Namespace moviemate: agrupa y aísla todos los recursos del proyecto de otros workloads del clúster, facilitando la gestión de permisos y la limpieza del entorno.
- ConfigMap: almacena la configuración no sensible de la aplicación, como el nombre de la base de datos, el perfil activo de Spring Boot, el puerto de escucha y la URL del servidor.
- Secret: almacena las credenciales sensibles codificadas en base64: contraseña de PostgreSQL, clave secreta JWT y clave de la API de TMDB. Kubernetes inyecta estos valores como variables de entorno en los contenedores en tiempo de arranque.
- PersistentVolumeClaim (PVC): solicita almacenamiento persistente para los datos de PostgreSQL, garantizando que la información se conserva entre reinicios de pod o actualizaciones del Deployment.
- Deployment de PostgreSQL: define el pod de base de datos, monta el PVC como volumen y carga las credenciales desde el Secret.
- Service ClusterIP para PostgreSQL: expone la base de datos internamente dentro del clúster mediante un nombre de servicio estable, desacoplando la dirección IP del pod de la configuración del back-end.
- Deployment del back-end: define el pod de la aplicación Spring Boot con la imagen Docker generada en el pipeline de CI/CD. Incluye sondas de liveness y readiness para garantizar que el servicio únicamente recibe tráfico cuando está completamente iniciado y operativo.
- Service NodePort para el back-end: expone el back-end externamente para permitir el acceso desde el navegador durante el desarrollo local con Minikube.

El proceso de despliegue local con Minikube sigue los pasos a continuación:

- Inicialización del clúster: minikube start arranca el nodo local y configura el contexto de kubectl para apuntar al clúster Minikube.
- Carga de la imagen: minikube image load moviemate-backend:latest inyecta la imagen Docker directamente en el registro interno del clúster, evitando la necesidad de un registro externo en el entorno de desarrollo.
- Aplicación de manifestos: kubectl apply -f k8s/ aplica todos los recursos definidos en el directorio de manifestos. Kubernetes gestiona automáticamente el orden de creación respetando las dependencias entre recursos.
- Verificación: kubectl get pods -n moviemate permite monitorizar el estado de los pods y confirmar que todos se encuentran en estado Running.

- Acceso al servicio: `minikube service backend-service -n moviemate` abre el servicio en el navegador, resolviendo automáticamente la IP del nodo y el puerto NodePort asignado.

En el entorno de integración continua, una vez que el pipeline de GitHub Actions construye y publica la imagen Docker con éxito tras un merge a main, se ejecuta automáticamente un paso adicional de despliegue que actualiza el Deployment del clúster mediante `kubectrl set image` o `kubectrl apply`. Esto garantiza que el clúster refleja siempre la última versión estable de la aplicación sin intervención manual.

La combinación de Minikube para el entorno local y Kubernetes para el entorno de producción garantiza la paridad entre ambos entornos, reduciendo el riesgo de diferencias de comportamiento entre el desarrollo y la puesta en producción.

[Figura 4.11: Diagrama del pipeline completo de CI/CD y despliegue automatizado en Kubernetes con Minikube, desde el push a la rama main hasta el pod en ejecución.]

5 Conclusiones y vías futuras

5.1 Conclusiones

El desarrollo de MovieMate ha permitido construir una plataforma web completa para la gestión y valoración de contenido audiovisual, cumpliendo con los objetivos planteados al inicio del proyecto. Se ha implementado una aplicación full-stack funcional que integra de forma coherente un back-end basado en Spring Boot y una interfaz de usuario desarrollada con React y TypeScript, desplegable en un entorno contenerizado mediante Docker y con un pipeline de integración continua operativo.

Desde el punto de vista técnico, el proyecto ha supuesto la aplicación práctica de un conjunto amplio de tecnologías y patrones propios del desarrollo web moderno. La arquitectura en capas adoptada en el back-end ha demostrado ser una elección acertada para un proyecto de esta escala, favoreciendo la separación de responsabilidades y la mantenibilidad del código. La estrategia de sincronización bajo demanda con la API de TMDB ha permitido reducir la dependencia de servicios externos y optimizar el rendimiento del sistema, almacenando el contenido únicamente cuando existe interacción real por parte de los usuarios.

En el ámbito del cliente, el uso de TanStack Query y Zustand ha simplificado considerablemente la gestión del estado asíncrono, reduciendo la complejidad asociada al desarrollo de aplicaciones de página única con múltiples fuentes de datos. La adopción de un sistema de diseño consistente ha resultado en una interfaz cohesiva, responsive y con una experiencia de usuario fluida, alineada con las expectativas de los usuarios de plataformas de referencia como Letterboxd o IMDb.

La incorporación de pruebas unitarias automatizadas sobre la capa de servicios y de un pipeline de integración continua ha reforzado la calidad del sistema, garantizando que las modificaciones al código son validadas de forma automática antes de su incorporación al repositorio principal.

5.2 Trabajo futuro

Aunque el sistema desarrollado cubre los objetivos planteados, existen diversas líneas de mejora y extensión que podrían incorporarse en futuras iteraciones del proyecto. En primer lugar, sería conveniente extender el soporte multiplataforma mediante el desarrollo de una aplicación móvil nativa para iOS y Android, aprovechando tecnologías como React Native que permitirían reutilizar la lógica de negocio ya desarrollada en el cliente web.

En segundo lugar, el motor de recomendaciones de la plataforma podría mejorarse incorporando algoritmos de filtrado colaborativo o técnicas de aprendizaje automático basadas en el historial de valoraciones, los géneros preferidos y la actividad social del usuario, con el objetivo de ofrecer sugerencias de contenido más personalizadas y relevantes.

En tercer lugar, la integración con plataformas de streaming permitiría mostrar en tiempo real la disponibilidad de cada título en los servicios a los que el usuario está suscrito, añadiendo un valor diferencial significativo. Asimismo, la incorporación de funcionalidades adicionales como

la planificación de visualizaciones en grupo, la gamificación de la experiencia o la exportación del historial de visionado ampliarían considerablemente el atractivo de la plataforma.

Desde el punto de vista de la escalabilidad, una vez que el proyecto alcanzase un volumen de usuarios significativo, podría estudiarse la transición hacia una arquitectura de microservicios, desacoplando los servicios de mayor carga (como las notificaciones en tiempo real o la integración con TMDb) en unidades independientes con escalado horizontal autónomo.

Puede estar subdividido en dos apartados: (1) conclusiones, (2) trabajo futuro. El trabajo futuro se refiere a carencias actuales del trabajo que están sujetas a mejoras en el futuro, posibles extensiones para dotar de mayor funcionalidad, etc.

6 Bibliografía¹

- [1] Nielsen. (2024). *El tiempo dedicado al streaming supera el 40 % en junio, la mayor cuota de uso de la televisión en la historia de The Gauge™ de Nielsen*. [Online]. Available: <https://www.nielsen.com/es/news-center/2024/time-spent-streaming-surges-to-over-40-percent-in-june-2024/>
- [2] ADSLZone. (2025). *Mejores apps para cinéfilos: listas de películas por ver*. [Online]. Available: <https://www.adslzone.net/noticias/moviles/apps-control-peliculas-vistas-por-ver/>
- [3] Robert Steiner. (2025). Why Letterboxd became the go-to platform for online film buffs. Variety. [Online]. Available: <https://variety.com/vip/letterboxd-year-end-report-growth-1236277320/>
- [4] Letterboxd. (2024). *Letterboxd - Social film discovery*. [Online]. Available: <https://letterboxd.com/>
- [5] IMDb. (2024). *IMDb - Ratings, Reviews, and Where to Watch the Best Movies & TV Shows*. [Online]. Available: <https://www.imdb.com/>
- [6] Serializd. (2024). *Serializd - The social platform for TV*. [Online]. Available: <https://serializd.com/>
- [7] Spring. (2024). *Spring Boot documentation*. [Online]. Available: <https://spring.io/projects/spring-boot>
- [8] Oracle. (2024). *Java Platform, Standard Edition*. [Online]. Available: <https://www.java.com/>
- [9] PostgreSQL Global Development Group. (2024). *PostgreSQL Database*. [Online]. Available: <https://www.postgresql.org/>
- [10] Amazon Web Services. (2024). ¿Qué es una API de RESTful?. [Online]. Available: <https://aws.amazon.com/es/what-is/restful-api/>
- [11] OpenAPI Initiative. (2025). *OpenAPI Specification – Define standard, language-agnostic RESTful APIs*. [Online]. Available: <https://www.openapis.org/>
- [12] Swagger. (2025). *API Development Tools*. [Online]. Available: <https://swagger.io/>
- [13] H2 Database Engine. (2025). *H2 – Java SQL Database*. [Online]. Available: <https://www.h2database.com/>
- [14] JaCoCo. (2025). *JaCoCo – Java Code Coverage Library*. [Online]. Available: <https://www.jacoco.org/jacoco/>
- [15] Meta Open Source. (2024). *React – A JavaScript library for building user interfaces*. [Online]. Available: <https://react.dev/>
- [16] Microsoft. (2024). *TypeScript – Typed JavaScript at Any Scale*. [Online]. Available: <https://www.typescriptlang.org/>
- [17] Tailwind Labs. (2024). *Tailwind CSS Documentation*. [Online]. Available: <https://tailwindcss.com/>
- [18] Mantine. (2024). *Mantine React Components Library*. [Online]. Available: <https://mantine.dev/>

Field Code Changed

Field Code Changed

Field Code Changed

Field Code Changed

Field Code Changed

Field Code Changed

¹ https://biblioguias.uam.es/citar/estilo_ieee

[19] The Movie Database (TMDB). (2024). *TMDB API and documentation*. [Online]. Available: <https://www.themoviedb.org/>

[20] Microsoft. (2024). *Visual Studio Code*. [Online]. Available: <https://code.visualstudio.com/>

[21] Git SCM. (2024). *Git – Distributed version control system*. [Online]. Available: <https://git-scm.com/>

[22] GitHub. (2024). *GitHub – Where the world builds software*. [Online]. Available: <https://github.com/>

[23] Postman. (2024). *Postman API Platform*. [Online]. Available: <https://www.postman.com/>

[24] pgAdmin Development Team. (2025). *pgAdmin 4 – PostgreSQL Tools and Management*. [Online]. Available: <https://www.pgadmin.org/>

[25] DBeaver Corp. (2025). *DBeaver – Universal Database Tool*. [Online]. Available: <https://dbeaver.com/>

[26] OpenAI. (2024). *ChatGPT*. [Online]. Available: <https://chatgpt.com/>

[27] JUnit Team. (2025). *JUnit 5 User Guide*. [Online]. Available: <https://junit.org/junit5/>

[28] Mockito Contributors. (2025). *Mockito – Tasty mocking framework for unit tests in Java*. [Online]. Available: <https://site.mockito.org/>

[29] Docker Inc. (2025). *Docker – Accelerated Container Application Development*. [Online]. Available: <https://www.docker.com/>

[30] Docker Inc. (2025). *Docker Compose – Define and run multi-container applications*. [Online]. Available: <https://docs.docker.com/compose/>

[31] GitHub. (2025). *GitHub Actions – Automate your workflow*. [Online]. Available: <https://github.com/features/actions>

Normalmente se organiza de una de las siguientes dos formas: (1) ordenado por orden de aparición en el documento; (2) ordenado alfabéticamente por el apellido del primer autor. ¡Todas las entradas en el apartado de bibliografía tienen que aparecer referenciadas en el texto!

Anexo A – Especificación de casos de uso

Este anexo recoge la especificación formal de los casos de uso principales del sistema MovieMate. Cada caso de uso se documenta utilizando una plantilla estructurada que describe el actor implicado, la funcionalidad del sistema, el objetivo perseguido y el resultado esperado.

Esta documentación complementa los diagramas presentados en el capítulo 4.1, proporcionando mayor detalle sobre el comportamiento funcional del sistema.

A.1 Usuarios no registrados

Caso de uso: Consultar catálogo de contenido

Actor: Usuario no registrado

Descripción: Permite visualizar el catálogo general de películas y series disponibles en la plataforma.

Objetivo: Explorar el contenido disponible sin necesidad de autenticación.

Resultado: El sistema muestra el catálogo actualizado de películas y series.

Caso de uso: Consultar ficha de contenido

Actor: Usuario no registrado

Descripción: Permite visualizar la información detallada de una película o serie concreta.

Objetivo: Obtener detalles específicos de un contenido seleccionado.

Resultado: El sistema muestra la ficha completa del contenido.

Caso de uso: Explorar contenido destacado

Actor: Usuario no registrado

Descripción: Permite visualizar listados de contenido popular o recomendado.

Objetivo: Facilitar el descubrimiento de contenido relevante.

Resultado: El sistema muestra contenidos destacados.

Caso de uso: Visualizar perfil público

Actor: Usuario no registrado

Descripción: Permite consultar la información pública de otro usuario.

Objetivo: Conocer la actividad pública y listas visibles de otros usuarios.

Resultado: El sistema muestra el perfil público solicitado.

Caso de uso: Consultar listas públicas

Actor: Usuario no registrado

Descripción: Permite visualizar listas públicas creadas por otros usuarios.

Objetivo: Explorar contenido organizado por la comunidad.

Resultado: El sistema muestra la lista pública seleccionada.

Caso de uso: Registrarse

Actor: Usuario no registrado

Descripción: Permite crear una cuenta en la plataforma.

Objetivo: Obtener acceso a funcionalidades avanzadas.

Resultado: El sistema registra un nuevo usuario.

Caso de uso: Iniciar sesión

Actor: Usuario registrado

Descripción: Permite autenticarse en el sistema.

Objetivo: Acceder a funcionalidades personalizadas.

Resultado: El sistema valida las credenciales y concede acceso.

A.2 Interacción social

Caso de uso: Gestionar perfil de usuario

Actor: Usuario registrado

Descripción: Permite modificar la información personal del perfil.

Objetivo: Mantener actualizados los datos del usuario.

Resultado: El sistema guarda los cambios realizados.

Caso de uso: Configurar privacidad del perfil

Actor: Usuario registrado

Descripción: Permite definir si el perfil del usuario es público o privado.

Objetivo: Controlar quién puede visualizar la información del perfil.

Resultado: El sistema actualiza la configuración de privacidad.

Caso de uso: Seguir usuario

Actor: Usuario registrado

Descripción: Permite seguir a otro usuario de la plataforma con perfil público.

Objetivo: Recibir actualizaciones de actividad de otros usuarios.

Resultado: El sistema registra la relación de seguimiento.

Caso de uso: Solicitar seguimiento

Actor: Usuario registrado

Descripción: Permite solicitar seguir a un usuario con perfil privado.

Objetivo: Obtener acceso al contenido público y la actividad del usuario.

Resultado: El sistema registra la solicitud y genera una notificación.

Caso de uso: Aceptar solicitud de seguimiento

Actor: Usuario registrado

Descripción: Permite aprobar una solicitud de seguimiento recibida.

Objetivo: Autorizar el acceso al contenido permitido.

Resultado: El sistema establece la relación de seguimiento.

Caso de uso: Rechazar solicitud de seguimiento

Actor: Usuario registrado

Descripción: Permite denegar una solicitud de seguimiento.

Objetivo: Mantener la privacidad del perfil.

Resultado: El sistema elimina la solicitud pendiente.

Caso de uso: Dejar de seguir usuario

Actor: Usuario registrado

Descripción: Permite eliminar una relación de seguimiento.

Objetivo: Dejar de recibir actividad de otro usuario.

Resultado: El sistema elimina la relación de seguimiento.

Caso de uso: Consultar seguidores y seguidos

Actor: Usuario registrado

Descripción: Permite visualizar las relaciones sociales del usuario.

Objetivo: Conocer la red de conexiones dentro de la plataforma.

Resultado: El sistema muestra seguidores y usuarios seguidos.

Caso de uso: Consultar feed de actividad

Actor: Usuario registrado

Descripción: Permite visualizar la actividad reciente de usuarios seguidos.

Objetivo: Mantenerse informado sobre novedades sociales.

Resultado: El sistema muestra el feed personalizado.

Caso de uso: Consultar notificaciones

Actor: Usuario registrado

Descripción: Permite visualizar notificaciones relacionadas con actividad social.

Objetivo: Mantenerse informado de interacciones.

Resultado: El sistema muestra el listado de notificaciones.

Caso de uso: Marcar notificaciones como leídas

Actor: Usuario registrado

Descripción: Permite indicar que una notificación ha sido revisada.

Objetivo: Gestionar el estado de las notificaciones.

Resultado: El sistema actualiza el estado de lectura.

A.3 Listas personalizadas

Caso de uso: Crear lista personalizada

Actor: Usuario registrado

Descripción: Permite crear una lista de contenido.

Objetivo: Organizar películas y series según criterios personales.

Resultado: El sistema registra la nueva lista.

Caso de uso: Editar lista personalizada

Actor: Usuario registrado

Descripción: Permite modificar una lista existente.

Objetivo: Actualizar información o visibilidad de la lista.

Resultado: El sistema guarda los cambios realizados.

Caso de uso: Eliminar lista personalizada

Actor: Usuario registrado

Descripción: Permite borrar una lista propia.

Objetivo: Eliminar contenido organizado que ya no es necesario.

Resultado: El sistema elimina la lista.

Caso de uso: Añadir contenido a lista

Actor: Usuario registrado

Descripción: Permite incluir películas o series en una lista.

Objetivo: Organizar contenido visualizado o pendiente.

Resultado: El sistema añade el contenido a la lista.

Caso de uso: Eliminar contenido de lista

Actor: Usuario registrado

Descripción: Permite retirar contenido de una lista.

Objetivo: Mantener la lista actualizada.

Resultado: El sistema elimina el contenido seleccionado.

Caso de uso: Consultar listas del propio usuario

Actor: Usuario registrado

Descripción: Permite visualizar listas creadas por el usuario.

Objetivo: Visualizar las listas del usuario.

Resultado: El sistema muestra las listas del usuario.

Caso de uso: Consultar listas públicas

Actor: Usuario registrado

Descripción: Permite visualizar listas públicas de otros usuarios a los que sigue el usuario propio.

Objetivo: Visualizar las listas públicas de otros usuarios.

Resultado: El sistema muestra las listas públicas de usuarios a los que el usuario sigue.

Caso de uso: Comentar valoración

Actor: Usuario registrado

Descripción: Permite publicar un comentario sobre la reseña de otro usuario.

Objetivo: Fomentar la interacción social en torno a listas públicas.

Resultado: El sistema registra el comentario y una notificación al propietario de la lista.

A.4 Valoraciones y reseñas

Caso de uso: Crear valoración

Actor: Usuario registrado

Descripción: Permite valorar una película o serie.

Objetivo: Expresar opinión sobre contenido.

Resultado: El sistema registra la valoración.

Caso de uso: Editar valoración

Actor: Usuario registrado

Descripción: Permite modificar una valoración previa.

Objetivo: Actualizar opinión registrada.

Resultado: El sistema guarda la modificación.

Caso de uso: Eliminar valoración

Actor: Usuario registrado

Descripción: Permite borrar una valoración.

Objetivo: Retirar una opinión registrada.

Resultado: El sistema elimina la valoración.

Caso de uso: Consultar valoraciones propias

Actor: Usuario registrado

Descripción: Permite visualizar valoraciones creadas por el usuario.

Objetivo: Visualizar las opiniones publicadas.

Resultado: El sistema muestra valoraciones del usuario.

Caso de uso: Consultar valoraciones de contenido

Actor: Usuario registrado

Descripción: Permite visualizar valoraciones de otros usuarios.

Objetivo: Conocer opiniones de la comunidad.

Resultado: El sistema muestra valoraciones disponibles.

Caso de uso: Dar me gusta a valoración

Actor: Usuario registrado

Descripción: Permite indicar interés por una reseña.

Objetivo: Destacar contenido útil.

Resultado: El sistema registra la interacción y genera notificación.

Resultado: El sistema registra la interacción y genera notificación.

Caso de uso: Comentar valoración

Actor: Usuario registrado

Descripción: Permite publicar un comentario sobre la reseña de otro usuario.

Objetivo: Enriquecer el debate en torno a una valoración.

Resultado: El sistema registra el comentario y genera una notificación al autor de la reseña.